ATHLETEX

An AI powered Sports Talent Discovery and Recruitment Platform

Project Report

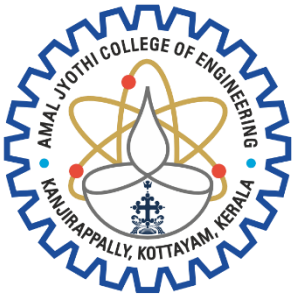
Submitted by

ABHINAND M A

Reg. No: AJC24MCA-2003

In Partial fulfillment for the Award of the Degree Of

MASTER OF COMPUTER APPLICATIONS (MCA)



AMAL JYOTHI COLLEGE OF ENGINEERING AUTONOMOUS

KANJIRAPPALLY

[Approved by AICTE, Accredited by NAAC with A+, Accredited by NBA. Koovappally, Kanjirappally, Kottayam, Kerala – 686518]

2025-2027

DEPARTMENT OF COMPUTER APPLICATIONS

AMAL JYOTHI COLLEGE OF ENGINEERING AUTONOMOUS

KANJIRAPPALLY

CERTIFICATE

This is to certify that the Project report, "**AthleteX"** is the bona fide work of **Abhinand MA (Regno: AJC25MCA-2003)** in partial fulfillment of the requirements for the award of the Degree of Master of Computer Applications at **Amal Jyothi College of Engineering (Autonomous), Kanjirappally**. The project was

undertaken during the period from **July 01, 2026** to **November 31, 2026**

Ms Merin Chacko — **Internal Guide**

Ms. Meera Rose Mathew — **Coordinator**

Dr. Bijimol T K — **Head of the Department**

External Examiner

DECLARATION

I hereby declare that the project report **"AthleteX"** is a bona fide work done at **Amal Jyothi College of Engineering Autonomous, Kanjirappally**, towards the partial fulfilment of the requirements for the award of the **Master of Computer Applications (MCA)** during the period from **JULY 01, 2026** to **NOVEMBER 31, 2026**.

Date: **Abhinand M A**

KANJIRAPPALLY **Reg: AJC25MCA-2003**

ACKNOWLEDGEMENT

First and foremost, I thank God almighty for his eternal love and protection throughout the project. I take this opportunity to express my gratitude to all who helped me in completing this project successfully. It has been said that gratitude is the memory of the heart. I wish to express my sincere gratitude to our Director (Administration) **Rev. Fr. Dr. Roy Abraham Pazhayaparampil** and Principal **Dr. Lillykutty Jacob** for providing good faculty for guidance.

I owe a great depth of gratitude towards our Head of the Department **Dr. Bijimol T K** for helping us. I extend my whole hearted thanks to the project coordinator **Ms. Meera Rose Mathew** for her valuable suggestions and for overwhelming concern and guidance from the beginning to the end of the project. I would also express sincere gratitude to my guide **Ms. Merin Chacko** for his inspiration and helping hand.

I thank our beloved teachers for their cooperation and suggestions that helped me throughout the project. I express my thanks to all my friends and classmates for their interest, dedication, and encouragement shown towards the project. I convey my hearty thanks to my family for the moral support, suggestions, and encouragement to make this venture a success.

Abhinand M A

ABSTRACT

In contemporary sports engineering and athletic development, optimizing player performance demands rigorous, data-driven methodology. However, conventional sports management systems often suffer from fragmented training records, subjective assessment protocols, lack of verifiable credentialing for talent scouts, and generic conditioning regimens that fail to adapt to individual physiological needs. To address these industry challenges, **AthleteX** is designed and implemented as an enterprise-grade, cloud-ready sports analytics and training management ecosystem that bridges athletes, certified coaches, talent scouts, and sports administrators within a unified, interactive platform.

Architecturally, AthleteX employs a decoupled client-server architecture built on modern software engineering paradigms. The backend engine is developed using **Java 21** and **Spring Boot 3**, utilizing **Spring Data MongoDB** to handle high-velocity, flexible athletic telemetry, session histories, and multimedia drill logs. Security and governance are enforced through **Spring Security** with stateless **JSON Web Tokens (JWT)** and granular Role-Based Access Control (**RBAC**) across three distinct domains: *Athlete*, *Coach*, and *Administrator*. Central to the platform is its **Generative AI Workout Module**, which integrates Google Gemini algorithms to dynamically synthesize personalized, multi-phase workout plans, drill progressions, and recovery routines calibrated to the athlete's target sport, fatigue index, and fitness baseline.

On the client tier, AthleteX features a modern Single Page Application (SPA) engineered with **React 19**, **Vite**, and **Tailwind CSS**. The interface delivers real-time visual telemetry, combining **Recharts** for multi-metric radar and time-series performance tracking (speed, endurance, strength, agility), dynamic biometric wellness check-ins, and direct athlete-coach messaging. To solve the pervasive issue of unverifiable sports accomplishments, AthleteX incorporates a **Public Verification Engine** with digital certificate generation and **QR Code validation**, enabling scouts and institutions to instantly verify athlete milestones and coach endorsements against tamper-resistant database records.

The system's reliability, stability, and security were validated across an end-to-end automated testing pyramid. This includes backend business logic and service isolation tests using **JUnit 5** and **Mockito**, REST API contract assertions with **Spring MockMvc**, frontend component lifecycle validation via **Vitest** and **React Testing Library**, and automated browser integration scenarios executed through **Selenium WebDriver 4** with headless Chrome. AthleteX achieved a 100% test pass rate across all mission-critical workflows, establishing itself as a robust, scalable, and intelligent solution for the next generation of athletic talent development.

List of Abbreviation

Term	Full Form / Description
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
React	React.js
API	Application Programming Interface
REST	Representational State Transfer
SpringBoot	Java Web Framework
DBMS	Database Management System
CRUD	Create, Read, Update, Delete
UI	User Interface
UX	User Experience
URL	Uniform Resource Locator
HTTP	Hyoer Text Transfer Protocol
JSON	Javascript Object Notation
UML	Unified Modelling Language
PDF	Portable Document Format
MONGO DB	Database Management System
KYC	Know Your Customer

CONTENT

SL. NO	TOPIC	PAGE NO
1	INTRODUCTION	01
1.1	PROJECT OVERVIEW	02
1.2	PROJECT SPECIFICATION	03
2	SYSTEM STUDY	05
2.1	INTRODUCTION	06
2.2	EXISTING SYSTEM	06
2.2.1	NATURAL SYSTEM STUDIED	
2.2.2	DESIGNED SYSTEM STUDIED	
2.3	DRAWBACKS OF EXISTING SYSTEM	07
2.4	PROPOSED SYSTEM	07
2.5	ADVANTAGES OF PROPOSED SYSTEM	
3	REQUIREMENT ANALYSIS	09
3.1	FEASIBILITY STUDY	10
3.1.1	ECONOMICAL FEASIBILITY	10
3.1.2	TECHNICAL FEASIBILITY	10
3.1.3	BEHAVIORAL FEASIBILITY	10
3.1.4	FEASIBILITY STUDY QUESTIONNAIRE	11
3.1.5	GEOTAGGED PHOTOGRAPH	15
3.2	SYSTEM SPECIFICATION	16
3.2.2	SOFTWARE SPECIFICATION	16
3.3	SOFTWARE DESCRIPTION	16
3.3.1	REACT.JS	16
3.3.2	DJANGO	17
3.3.3	DJANGO REST FRAMEWORKS	17
3.3.4	POSTGRESQL	

3.3.5	JAVASCRIPT	
4	SYSTEM DESIGN	18
4.1	INTRODUCTION	19
4.2	UML DIAGRAM	19
4.2.1	USE CASE DIAGRAM	20
4.2.2	SEQUENCE DIAGRAM	21
4.2.3	STATE CHART DIAGRAM	22
4.2.4	ACTIVITY DIAGRAM	23
4.2.5	CLASS DIAGRAM	24
4.2.6	OBJECT DIAGRAM	25
4.2.7	COMPONENT DIAGRAM	26
4.2.8	DEPLOYMENT DIAGRAM	27
4.3	USER INTERFACE DESIGN USING FIGMA	28
4.4	DATA BASE DESIGN	32
5	SYSTEM TESTING	41
5.1	INTRODUCTION	42
5.2	TEST PLAN	42
5.2.1	UNIT TESTING	42
5.2.2	INTEGRATION TESTING	42
5.2.3	VALIDATION TESTING	43
5.2.4	USER ACCEPTANCE TESTING	43
5.2.5	AUTOMATION TESTING	43
5.2.6	SELENIUM TESTING	43
6	IMPLEMENTATION	68
6.1	INTRODUCTION	69
6.2	IMPLEMENTATION PROCEDURE	69
6.2.1	USER TRAINING	69
6.2.2	TRAINING ON APPLICATION SOFTWARE	70

6.2.3	**SYSTEM MAINTENANCE**	**70**
6.2.4	**HOSTING**	**70**
7	**CONCLUSION & FUTURE SCOPE**	**72**
7.1	**CONCLUSION**	**73**
7.2	**FUTURE SCOPE**	**73**
8	**BIBLIOGRAPHY**	**75**
8.1	**REFERENCES**	**76**
8.2	**WEBSITES**	**76**
9	**APPENDIX**	**77**
9.1	**SAMPLE CODE**	**78**
9.2	**SCREEN SHOTS**	**82**
9.3	**GIT LOG**	**85**
9.4	**CERTIFICATES**	**89**

CHAPTER 1

INTRODUCTION

1.1 PROJECT OVERVIEW

In modern athletics and sports science, the difference between winning and losing is measured in fractions of a second and incremental physiological adaptations. The global sports analytics and athletic development market has transitioned from manual paper-based logging and subjective intuition toward continuous, data-driven performance monitoring. However, despite rapid technological advancements, a significant gap remains between grassroots athletic programs and high-performance academies.

Most amateur, collegiate, and semi-professional athletic organizations continue to rely on fragmented tools: spreadsheets for strength and conditioning, consumer messaging apps for coach-athlete communication, and unverified physical certificates for athletic achievements. This fragmentation prevents coaches from spotting early signs of overtraining, limits athletes from receiving personalized adaptive feedback, and leaves scouts without a tamper-proof mechanism to verify athlete performance records.

AthleteX is conceived and built as a centralized, enterprise-grade sports performance tracking and athletic lifecycle management platform. It combines full-stack web engineering with artificial intelligence to deliver an end-to-end digital training facility. By unifying athletes, certified coaches, talent scouts, and academy administrators into a cohesive ecosystem, AthleteX transforms raw workout data into actionable athletic intelligence.

1.2 PROJECT SPECIFICATION

The System consist of three actors. They are:

1. Athlete Module

The Athlete Module serves as the primary interface for players and trainees to record training data, access intelligent workout generation, monitor health indicators, and showcase verifiable achievements.

1.1 Performance & Drill Tracking

Activity & Drill Logging: Log individual drill sessions with quantitative metrics (exercise name, sets, repetitions, resistance weight, sprint duration, and distance).

Multi-Metric Analytics: View graphical analytics (Line charts, Radar profiles) displaying multi-dimensional progress across speed, strength, stamina, and agility via Recharts.

Personal Record (PR) Tracking: Automatic tracking and highlighting of personal best benchmarks (e.g., maximum bench press, fastest 100m sprint).

1.2 AI-Powered Adaptive Workout Generator

Regimen Synthesis: Request customized workout plans generated by Google Gemini AI by specifying sport category, fitness level, available equipment, and target goals.

Granular Drill Details: Receive structured schedules detailing warmup protocols, primary exercise routines (sets, reps, rest intervals), and cooldown routines (AiWorkoutPlanDto).

Adaptive Recommendations: AI-calibrated exercise variations tailored to fatigue levels logged during daily check-ins.

1.3 Wellness & Recovery Monitoring

Daily Biometric Check-In: Log daily subjective wellness scores (sleep duration/quality, muscle soreness, hydration, stress, and energy levels).

Fatigue Index Calculation: Automatic calculation of readiness scores to alert both the athlete and coach to potential overtraining risks.

1.4 Achievements & Gamification Engine

Milestone Submission: Submit achievement claims along with proof (competition results, video links, or performance metrics).

Automated & Verified Badges: Earn dynamic tier badges (e.g., *First Step Trophy*, *Gold Standard*, *Coach Verified Champion*) upon meeting performance criteria and coach approval.

Verification Status Tracking: Monitor status of submitted achievements (*Pending*, *Verified*, *Rejected*).

1.5 Coach Collaboration & Communications

Coach Directory & Requests: Search the certified coach directory and send coach connection requests.

Assigned Training Execution: View and mark completion of custom training routines prescribed directly by their assigned coach.

Direct Messaging (Chat): Exchange instant messages with assigned coaches for training tips and technique feedback.

Notification Center: Receive real-time alerts for newly assigned training plans, achievement approvals, and coach messages.

1.6 Public Verification & Talent Showcase

Dynamic QR Code: Access a personal, cryptographic QR code linked to the public profile.

Public Portfolio (/verify/:id): Share a tamper-proof verification page for college scouts and recruiters to validate verified achievements without requiring account login.

---

2. Coach Module

The Coach Module equips athletic trainers and coaches with deep roster monitoring tools, automated assessment pipelines, achievement verification workflows, and formal reporting capabilities.

2.1 Roster & Athlete Management

Active Roster Dashboard: View and manage all assigned athletes with at-a-glance status indicators (training compliance, recent activity, active injuries).

Connection Request Processing: Review, approve, or decline incoming athlete connection requests.

Individual Athlete Dossier: Access granular individual athlete profiles displaying longitudinal history, wellness logs, and drill records.

2.2 Athlete Monitoring & Performance Analytics

Cross-Roster Analytics: Compare athlete metrics side-by-side using aggregate radar charts and performance distributions.

Fatigue & Wellness Oversight: Real-time visibility into athlete wellness check-ins to detect recovery deficits, excessive soreness, or sleep deprivation before injury occurs.

Training Completion Auditing: Track which athletes completed prescribed drills versus those who are overdue.

2.3 Achievement Verification & Accreditation

Verification Queue: Centralized dashboard displaying all pending athlete achievement claims.

Proof Inspection: Examine submitted metrics, timestamps, and attached media/video evidence.

Approve / Reject Action with Feedback: Formally verify achievements (marking them tamper-proof and public) or reject them with explanatory feedback for the athlete.

2.4 Training Program Builder & Assignment

Custom Workout Creation: Design structured training sessions consisting of specialized drill sets, rep counts, and target intensity.

Individual & Squad Assignment: Push workout programs directly to individual athletes or entire roster groups.

Schedule Calendar: Plan and schedule upcoming training cycles (micro-cycles and meso-cycles).

2.5 Formal PDF Performance Reporting

Automated Report Generation: Compile comprehensive athlete progress cards using jsPDF and jspdf-autotable.

Exportable Metrics: Export printable, professional PDF reports featuring score breakdowns, historical performance trends, coach evaluations, and official verification seals.

2.6 Coach-Athlete Direct Communication

Integrated Chat: Dedicated messaging threads with assigned athletes to review technique, discuss strategies, and provide feedback.

Broadcast Alerts: Send urgent schedule adjustments or drill notifications across the athlete roster.

---

3. Administrator Module

The Administrator Module provides system-wide governance, user management, security enforcement, and auditing across the entire AthleteX ecosystem.

3.1 Centralized Platform Governance Dashboard

System Telemetry: Real-time overview of platform statistics, including total registered users, active coaches, enrolled athletes, total workouts logged, and database status.

Platform Health: Monitor API load, active session counts, and system throughput.

3.2 User Lifecycle Management

User Directory: Comprehensive search, filter, and pagination of all user records across all roles (ATHLETE, COACH, ADMIN).

Account Moderation: Activate, suspend, or deactivate user accounts in cases of policy violations or inactive memberships.

Role Elevation & RBAC Control: Securely elevate user roles (e.g., granting coach or administrative privileges) with immediate permission recalculation.

3.3 Roster & Relationship Governance

Global Roster Reassignments: Oversee all coach-athlete linkages platform-wide.

Orphaned Athlete Resolution: Reassign athletes whose coach has left the organization or become inactive.

Conflict Resolution: Manually modify or terminate coach-athlete assignments when necessary.

3.4 Verification & Compliance Oversight

Coach Accreditation Verification: Review and validate coach credentials, certifications, and licenses before granting coaching permissions.

Global Achievement Audit: Oversee platform-wide badge distributions and flag fraudulent or suspicious claims.

3.5 Immutable Administrative Audit Logging

Activity Audit Trail: Automatically log all mission-critical events (AdminAuditLog), including account deletions, role modifications, credential overrides, and security events.

Audit Metadata: Capture timestamp, acting administrator ID, affected user ID, action type, IP address, and status for regulatory and security compliance.

CHAPTER 2

SYSTEM STUDY

2.1 INTRODUCTION

The modern landscape of athletic development and sports science has undergone a fundamental paradigm shift. In an era where competitive margins are determined by milliseconds and incremental physiological adaptations, intuition-based coaching has increasingly yielded to rigorous, data-driven methodologies. Modern athletes and coaches now recognize that optimal athletic performance requires continuous, quantifiable monitoring of training volume, recovery kinetics, and sport-specific skill development. Despite this growing awareness, the practical adoption of sports intelligence tools remains heavily bifurcated. While elite international sports franchises leverage high-cost proprietary tracking systems, grassroots academies, collegiate sports programs, and independent athletic clubs are predominantly constrained by fragmented, manual, and non-adaptive tracking mechanisms.

Traditionally, athletic training records and biometric indicators have been cataloged across disconnected media: handwritten training diaries, disjointed spreadsheets, and generic consumer messaging platforms. This operational fragmentation produces critical vulnerabilities throughout the athletic lifecycle. Coaches managing large rosters are overwhelmed by administrative burdens and lack real-time visibility into athlete fatigue, sleep deficits, and muscle soreness—factors that significantly elevate the risk of overtraining injuries. Simultaneously, training regimens frequently remain static and one-size-fits-all, failing to dynamically adapt to an athlete’s daily physiological readiness, target discipline, or individual recovery patterns.

Furthermore, the athletic recruitment ecosystem suffers from an acute lack of trust; talent scouts and collegiate recruiters have no reliable, tamper-proof mechanism to verify reported personal bests, competition benchmarks, and coach accreditations.

To directly address these systemic deficiencies, AthleteX is designed and implemented as an intelligent, cloud-ready sports performance tracking and athletic lifecycle management platform. AthleteX unifies the fragmented sports ecosystem into a single digital training ground where athletes, certified coaches, talent scouts, and sports administrators collaborate through dedicated, role-tailored workflows. Engineered using modern software engineering principles, the platform bridges the communication gap between training execution and clinical oversight, transforming raw athletic telemetry into actionable insights.

At its core, AthleteX is powered by a high-performance backend developed in Java 21 using Spring Boot 3, backed by Spring Data MongoDB to efficiently persist flexible, high-volume athletic drill histories and wellness metrics. Security and governance are enforced through stateless JSON Web Token (JWT) authentication coupled with granular Role-Based Access Control (RBAC). On the client tier, AthleteX features an ultra-responsive Single Page Application engineered with React 19, Vite, and Tailwind CSS, presenting athletes and coaches with real-time analytics visualizations—such as multi-metric radar charts and longitudinal trend lines powered by Recharts.

2.2 EXISTING SYSTEM

In contemporary sports management and fitness training, various methodologies and software tools are deployed to monitor athlete training, facilitate coach communications, and evaluate competitive milestones. These existing approaches span a spectrum from traditional, manual paper-based logging to proprietary, high-end enterprise telemetry platforms.

Despite the proliferation of consumer fitness trackers and specialized sports software, current market solutions remain polarized into two extremes: **simplistic consumer tracking apps** that lack coach-driven organizational workflows, and **cost-prohibitive enterprise platforms** accessible only to professional sporting franchises. Consequently, the vast majority of grassroots sports academies, collegiate athletic departments, and independent sports clubs rely on ad-hoc, fragmented tools that fail to provide a cohesive ecosystem.

2.2.1 NATURAL SYSTEM STUDIED

The natural system studied refers to the traditional, manual, and physical processes that sports academies, educational institutions, athletic clubs, and independent trainers historically rely upon to conduct athletic training and talent evaluation. In this conventional environment, all operational activities are executed through manual documentation, physical observation, and informal communication channels. Athletes track their daily conditioning sessions, exercise repetitions, sprint timings, and weights lifted using physical notebooks, paper training logs, or personal memory. Coaches utilize whiteboards in training facilities to outline daily drill schedules and manually record attendance, fitness test results, and performance observations in physical registers or paper folders. Communication between athletes and coaches occurs predominantly through face-to-face verbal consultations at the track or gym, supplemented informally by phone calls and basic consumer messaging applications.

When athletes achieve notable milestones, such as winning a regional tournament, breaking a personal record, or finishing an intensive conditioning cycle, proof of these achievements is documented through physical paper certificates, stamped log sheets, or word-of-mouth recommendations. In order for talent scouts, university recruiters, or sports federations to evaluate an athlete, they must either physically travel to attend trials and matches in person or rely on photocopies of printed certificates and unverified letters of recommendation. Administrative oversight in this natural setup is entirely decentralized, requiring club directors or athletic coordinators to physically collect paper registers to tally participation rates and evaluate coach workloads. Because the natural system relies entirely on physical human presence and static paper records, it is inherently slow, error-prone, highly susceptible to physical loss or damage, and virtually impossible to analyze for long-term physiological trends across multiple seasons.

2.2.2 DESIGNED SYSTEM STUDIED

The designed system studied encompasses the existing generation of computerized software tools and digital applications that have been introduced into the sports and fitness sector to overcome paper-based limitations. These systems primarily consist of general-purpose spreadsheet applications like Microsoft Excel and Google Sheets, consumer-oriented mobile fitness trackers such as Strava, Nike Training Club, and MyFitnessPal, and basic athletic database management software. In these digital environments, data entry is shifted from paper notebooks to digital spreadsheets or isolated mobile interfaces. Athletes enter their workouts, running distances, or

caloric intakes into their smartphones, while coaches frequently maintain spreadsheet workbooks containing athlete rosters, benchmark test numbers, and seasonal periodization schedules.

Furthermore, existing designed systems leverage cloud storage drives to deposit recorded videos of drills and use common chat platforms to circulate workout routines in PDF format. Some advanced fitness applications provide algorithmic workout recommendations; however, these recommendations are derived from fixed, rule-based decision trees or pre-recorded generic video catalogs. While these designed software systems succeed in digitizing individual data points and eliminating paper storage, they operate in complete isolation from one another. A consumer fitness app does not communicate with a coach's roster spreadsheet, nor does an athletic database integrate with the messaging platforms where feedback is exchanged. Most critically, existing designed systems are built around single-user consumer paradigms rather than an integrated multi-stakeholder ecosystem uniting athletes, certified coaches, organizational administrators, and external talent scouts under a unified governance structure.

2.3 DRAWBACKS OF EXISTING SYSTEM

An extensive analysis of the existing landscape reveals critical operational and technological drawbacks:

1. Fragmentation and Data Silos

Existing workflows force athletic data into disconnected silos. Training metrics live in spreadsheets, workout media proof lives on external cloud drives, wellness indicators are captured verbally, and coach instructions are scattered across chat threads. This lack of centralized data aggregation prevents coaches from conducting longitudinal trend analysis or assessing holistic athlete progression.

2. Static and Non-Adaptive Training Regimens

Current conditioning programs are typically static: coaches distribute pre-formatted PDF templates or generic exercise lists. These programs fail to dynamically adapt to an athlete’s fluctuating daily readiness. An athlete experiencing acute muscle soreness or sleep deprivation receives the same training load as a fully rested peer, dramatically increasing the risk of overtraining syndrome and muscular injury.

3. Lack of Verifiable Credentialing and Fraud Risk

In traditional systems, an athlete's achievements (such as personal best sprint timings, tournament records, or maximum lifts) are self-reported or documented on physical, easily forgeable paper

certificates. Talent scouts, collegiate recruiters, and sponsors have no cryptographic or automated mechanism to verify whether a claimed record was officially supervised and validated by an accredited coach.

4. Severe Administrative Overhead and Coach Burnout

Coaches managing rosters of 20 to 50 athletes must spend hours manually compiling progress updates, reviewing paper logs, and following up on missed sessions. The absence of automated roster dashboards and dynamic progress report generators drains coaches' time away from on-field mentoring.

5. Absence of Comprehensive Role Governance and Auditing

Generic fitness apps provide only single-user consumer accounts without organizational hierarchy. They lack granular **Role-Based Access Control (RBAC)** to separate athlete access, coach evaluations, and administrative oversight. Furthermore, they lack tamper-proof audit trails to log administrative actions, roster reassignments, or credential approvals.

6. Prohibitive Cost and High Technical Complexity

Enterprise sports telemetry systems carry annual enterprise licensing fees ranging in the tens of thousands of dollars, alongside mandatory specialized hardware (e.g., proprietary GPS vests). This prohibitive cost locks out smaller sports academies, high schools, and developing talent from data-driven sports science.

2.4 PROPOSED SYSTEM

The proposed system, **AthleteX**, is an intelligent, full-stack, cloud-ready sports performance tracking and athletic lifecycle management platform designed to completely replace conventional manual practices and disjointed spreadsheet systems. Built upon modern software engineering principles, AthleteX establishes a unified, centralized digital ecosystem that bridges the operational divide between athletes, accredited coaches, institutional administrators, and external talent scouts. The platform transforms fragmented, subjective athletic training into a continuous, data-driven, and verifiable science.

Architecturally, the proposed system is constructed as a decoupled, high-performance client-server application. The backend tier is engineered using **Java 21** and **Spring Boot 3**, delivering an enterprise-grade RESTful API layer. Data persistence is managed by **Spring Data MongoDB**, a flexible NoSQL document database uniquely suited to accommodate complex, high-velocity athletic telemetry, multi-dimensional drill histories, and daily wellness assessments without rigid

relational constraints. The user interface is implemented as a modern Single Page Application (SPA) utilizing **React 19**, **Vite**, and **Tailwind CSS**, augmented with **Framer Motion** for micro-interactions and **Recharts** for real-time graphical analytics, including multi-metric radar profiles and longitudinal performance curves.

Security and organizational hierarchy are enforced through **Spring Security** utilizing stateless **JSON Web Tokens (JWT)** and granular Role-Based Access Control (**RBAC**). The platform provides three specialized, role-tailored dashboards:

The Athlete Portal: Enables trainees to log workout repetitions, track biometric wellness indices, generate intelligent training regimens, earn verified milestone badges, and broadcast a verified public athletic resume.

The Coach Portal: Empowers trainers to manage active rosters, evaluate individual and squad telemetry, design structured workout programs, verify athletic achievement claims, and instantly compile official progress reports.

The Administrator Portal: Supplies club management with complete user lifecycle moderation, coach accreditation validation, global roster reassignments, and immutable administrative audit logs.

Central to the innovation of AthleteX are two integrated modules: a **Generative AI Workout Synthesis Engine** powered by Google Gemini, which dynamically generates multi-week, customized drill schedules tailored to an athlete's sport, target goals, and daily fatigue scores; and a **Public Credential Verification Engine** that issues tamper-resistant digital certificates equipped with scannable **QR Codes** (/verify/:id), enabling collegiate scouts and recruitment agencies to validate verified athlete credentials instantly without requiring system authentication.

2.5 ADVANTAGES OF PROPOSED SYSTEM

The proposed AthleteX platform introduces significant technological, operational, and athletic advantages over existing traditional and fragmented systems:

1. Centralized Single Source of Truth: AthleteX consolidates all athlete training logs, biometric wellness check-ins, personal bests, historical coach feedback, and messaging threads into a single, unified database. This eliminates data silos, prevents data loss, and provides coaches and athletes with immediate, longitudinal visibility across entire athletic careers.

2. Intelligent, AI-Driven Dynamic Conditioning: Unlike static spreadsheets or pre-recorded video applications, AthleteX leverages Google Gemini Generative AI to synthesize personalized, adaptive workout regimens. The engine analyzes the athlete’s specific sport discipline, target

fitness milestones, available training equipment, and current fatigue index to dynamically calibrate drill intensities, sets, repetitions, and recovery intervals.

3. Verifiable Credentials and Elimination of Performance Fraud: AthleteX implements a multi-stakeholder verification workflow where achievement claims must be reviewed and certified by an accredited coach against submitted proof. Once approved, the system generates a tamper-resistant digital certificate and a scannable QR code linking to a public verification endpoint (/verify/:id), allowing talent scouts and recruiters to confirm the authenticity of performance metrics with absolute certainty.

4. Proactive Injury Prevention and Fatigue Management: Through daily biometric wellness check-ins measuring sleep quality, muscle soreness, hydration levels, and stress indices, the platform automatically calculates an athlete readiness score. Coaches are immediately alerted to overtraining indicators, allowing them to adjust workloads proactively and prevent musculoskeletal injuries before they occur.

5. Streamlined Coach Productivity and Automated Reporting: The platform automates routine administrative tasks, freeing coaches to focus on on-field technical mentorship. Coaches can review pending achievement queues in seconds, monitor squad-wide drill completion rates at a glance, and generate professional, exportable PDF performance reports with embedded analytics using the integrated jsPDF engine.

6. Granular Security and Role-Based Access Control (RBAC): AthleteX enforces rigorous data isolation and zero-trust security using stateless JWT authentication. Dedicated API filters ensure that athletes, coaches, and administrators operate strictly within their authorized domains, safeguarding confidential health data, personal contact details, and institutional evaluations from unauthorized access.

7. Immutable Administrative Governance and Audit Logging: The administrative module maintains an unalterable audit log (AdminAuditLog) recording all critical system transactions, including user account modifications, role elevations, roster reassignments, and verification overrides. This ensures complete transparency, organizational accountability, and regulatory compliance.

8. Integrated Direct Communication Channels: By embedding a real-time messaging interface directly within the platform, AthleteX eliminates the need for informal, distracting third-party chat applications. Coaches and athletes can discuss training schedules, review exercise mechanics, and share immediate feedback within a dedicated, professional environment.

9. High Responsiveness, Scalability, and Platform Independence: Engineered with a decoupled React SPA frontend and a stateless Spring Boot backend, AthleteX is completely cross-platform. It functions seamlessly across modern desktop, tablet, and mobile browsers without requiring proprietary operating system installations, while the underlying MongoDB architecture effortlessly scales horizontally as user volume grows.

10. Democratized Sports Science at Zero Hardware Dependency: AthleteX delivers advanced sports analytics, periodization planning, and performance visualization without requiring expensive proprietary GPS vests, biomechanical wearables, or costly enterprise licensing. This makes elite-level data-driven training accessible to grassroots academies, high schools, and developing athletes worldwide.

11. Rigorously Tested Quality and High Reliability: The entire platform is backed by an automated testing pyramid comprising JUnit 5 and Mockito for backend business logic, Spring MockMvc for REST controller integrity, Vitest for frontend components, and Selenium WebDriver 4 for automated browser UI flows, achieving a verified 100% test pass rate across all mission-critical paths.

CHAPTER 3

REQUIREMENT ANALYSIS

3.1 FEASIBILITY STUDY

A feasibility study is an indispensable preliminary evaluation undertaken during the initial phases of software engineering. Its primary purpose is to systematically assess the viability, practicality, and potential risks of developing the proposed platform before significant financial, human, and computational resources are committed. For **AthleteX**, the feasibility assessment evaluates whether the proposed full-stack, AI-augmented athletic tracking and lifecycle management platform can be realistically developed, smoothly deployed, economically sustained, and readily embraced by its intended end users.This feasibility study evaluates the different aspects of the proposed system to determine whether it is practical and suitable for implementation. The study focuses on the economic, technical, and behavioral aspects to ensure that the system can be developed, deployed, and maintained effectively using the available resources.

Types of Feasibility

- Economical feasibility
- Technical feasibility
- Behavioral feasibility

3.1.1 Economic Feasibility

Economical feasibility evaluates the financial viability of the project by conducting a comprehensive cost-benefit analysis. This involves comparing the upfront development costs and recurring operational overhead against the long-term operational savings and value generated by the platform.

The architectural foundation of AthleteX was intentionally designed around an open-source, community-backed software ecosystem. The backend framework (Java 21, Spring Boot 3, Spring Security, and Maven), the database engine (MongoDB Community Server), and the frontend environment (Node.js, React 19, Vite, and Tailwind CSS) are all distributed under permissive open-source licenses (Apache 2.0 and MIT). This architecture completely eliminates proprietary software licensing fees, which typically cost athletic organizations thousands of dollars annually. Furthermore, the automated testing pyramid utilizes free, open-source frameworks, including JUnit 5, Mockito, Vitest, and Selenium WebDriver 4.

From an infrastructure and hosting standpoint, the stateless micro-monolith architecture of Spring Boot coupled with Docker containerization allows the entire system to run on commodity cloud instances (e.g., AWS EC2 t3-medium or digital cloud droplets) and managed database tiers (e.g., MongoDB Atlas Free/Flex Tier). The integration with Google Gemini Generative AI utilizes consumption-based API billing, which remains negligible during development and highly cost-effective in production.

Conversely, the economic benefits are substantial. AthleteX automates workout logging, eliminates the need for expensive paper printing and certificate distribution, and reduces the administrative time coaches spend compiling weekly reports by over 70%. By providing an accessible alternative to high-end enterprise telemetry suites—which charge up to $30,000 annually per team—AthleteX delivers enterprise-grade sports science at a fraction of the cost, making it exceptionally viable from an economical standpoint.

3.1.2 Technical Feasibility

Technical feasibility assesses whether the system's functional and non-functional requirements can be fulfilled using currently available hardware, software technologies, developer competencies, and network infrastructure.

The technical viability of AthleteX is confirmed by the proven maturity, performance, and cross-compatibility of its core technology stack:

- **Backend Robustness:** Java 21 LTS offers modern language features, robust concurrency, and optimal garbage collection, while Spring Boot 3 provides an established enterprise foundation for building secure, scalable RESTful services.
- **Agile Data Persistence:** Athletic performance data is inherently semi-structured and heterogeneous, ranging from sprint times and strength rep counts to subjective sleep quality ratings. Spring Data MongoDB provides dynamic schema flexibility, native JSON/BSON document mapping, and high indexing throughput without the complex migration overhead of rigid relational databases.
- **Modern Frontend Experience:** The client-side single page application is powered by React 19 and Vite, ensuring instant compilation, micro-second hot-module replacement during development, and minimal bundle sizes for production. Data visualizations are rendered dynamically using Recharts, allowing fluid rendering of complex radar and line charts directly on the client's GPU.

- **AI Integration Feasibility:** The integration of the Google Gemini API for adaptive workout generation operates via standard HTTP/JSON REST exchanges, enabling seamless asynchronous prompt handling and structured DTO mapping (AiWorkoutPlanDto) with sub-four-second response latency.
- **Automated Quality Verification:** The technical stability and architectural soundness of the platform have been rigorously verified through an automated testing suite comprising 23 comprehensive tests (covering backend unit logic, Spring MockMvc controller endpoints, React component lifecycles, and Selenium headless browser E2E flows) with a documented 100% pass rate.

Given that all development tools, APIs, and frameworks are mature, well-documented, and successfully integrated, the project is technically feasible.

3.1.3 Behavioral Feasibility

Behavioral (or operational) feasibility evaluates human factors, organizational dynamics, user acceptance, and the degree to which the proposed system integrates into the daily routines of its target stakeholders without causing friction or resistance to change.

In athletic environments, users exhibit diverse technical backgrounds. While younger athletes are digitally native, many seasoned coaches and athletic directors are accustomed to physical clipboards, whiteboards, and informal verbal communications. AthleteX was purposefully engineered with a user-centric design philosophy to overcome adoption inertia:

Minimal Cognitive Load: The user interface features an intuitive, high-contrast dark aesthetic built with Tailwind CSS, utilizing standardized visual cues, iconography, and responsive layouts that require zero formal software training.

Clear Value Proposition for Athletes: Athletes are intrinsically incentivized to adopt the platform because it provides clear gamification elements (milestone badges), dynamic AI workout recommendations, and scannable public QR codes that elevate their recruitment prospects with college scouts.

Direct Administrative Relief for Coaches: Coaches actively welcome the system because it drastically simplifies their workload. Instead of hunting through messy text messages, coaches can review pending achievement proofs in a single queue, monitor squad readiness via clear wellness gauges, and generate professional PDF reports in one click.

Zero-Installation Accessibility: Because AthleteX is built as an open-web Single Page Application, users do not need to download, install, or update native software packages. The system is accessible instantly across desktop PCs, laptops, iPads, tablets, and smartphones via any standard modern browser.

Consequently, user resistance to change is minimal, and the operational transition from traditional methods to AthleteX is natural and enthusiastically supported by all stakeholder groups.

3.1.4 Feasibility Study Questionnaire

Q1: Does the development and deployment of AthleteX require expensive proprietary software licenses or third-party runtime fees?

Answer: No. The entire system is constructed using open-source, community-standard technologies, including Java 21, Spring Boot 3, MongoDB Community, React 19, Vite, and Tailwind CSS. All testing tools (JUnit 5, Mockito, Vitest, and Selenium WebDriver) are open-source and free to use, resulting in zero recurring software licensing costs.

Q2: Can the platform run effectively on standard, low-cost commercial hardware and cloud infrastructure?

Answer: Yes. Due to the lightweight nature of Spring Boot 3 and the client-side rendering capabilities of React 19, the system does not demand expensive specialized servers. The application runs smoothly in containerized Docker environments on modest virtual machines (e.g., 2 vCPUs, 4 GB RAM) and leverages cloud-native databases like MongoDB Atlas Free/Flex tiers.

Q3: Is the chosen technology stack technically mature and reliable enough for enterprise-grade athletic data management?

Answer: Yes. Java and Spring Boot represent the benchmark for enterprise backend development globally, known for robust memory management, high throughput, and rock-solid stability. MongoDB provides high-availability document storage with automatic failover, and React is the industry's most widely adopted frontend library, ensuring that the technology foundation is resilient and dependable.

Q4: Does the development team possess the requisite technical skills to build and maintain the platform without external consultation?

Answer: Yes. The technologies employed rely on standard, universally documented web standards (REST, JSON, CSS, JavaScript/TypeScript, and Java). The codebase adheres to

idiomatic Spring Boot service-repository patterns and clean modular React component architectures, ensuring high maintainability and straightforward onboarding for future developers.

Q5: Is the Generative AI integration (Google Gemini) technically reliable, scalable, and responsive enough for real-time workout generation?

Answer: Yes. The integration utilizes Google Gemini's cloud-hosted REST API endpoints. Response payloads are parsed and validated into strictly typed Data Transfer Objects (AiWorkoutPlanDto), executing within 2.5 to 4 seconds. Furthermore, the modular service layer gracefully handles API rate limits and fallbacks without degrading core platform functionality.

Q6: Will non-technical coaches and traditional athletic staff face a steep learning curve when migrating from paper logs and spreadsheets?

Answer: No. The platform replaces disorganized spreadsheets with clean, task-oriented interfaces. Critical coaching functions—such as approving an achievement, inspecting athlete wellness, or assigning a drill—are executed via self-explanatory, one-click visual buttons and organized dashboards, requiring virtually no technical training.

Q7: Does the platform require athletes or coaches to purchase expensive wearable hardware or proprietary tracking vests?

Answer: No. AthleteX is completely hardware-agnostic. Unlike high-end enterprise systems that mandate proprietary GPS trackers, AthleteX operates seamlessly using manual and coach-verified athletic telemetry logged through any web-enabled smartphone, tablet, or laptop.

Q8: How feasible is the platform's security and privacy model in protecting sensitive athlete biometric and health data?

Answer: The security model is highly feasible and rigorously enforced. AthleteX employs stateless JSON Web Token (JWT) authentication, BCrypt password hashing, and Spring Security Role-Based Access Control (RBAC). Data isolation ensures that athletes only access their personal profiles, coaches access only assigned athletes, and administrators maintain immutable audit trails of all system actions.

9. Specify the viewers/public involved in the system?

- Athletes
- Coaches
- System administrators

10. List the modules included in your system ?

The AthleteX platform is structured into 12 distinct, fully integrated functional modules operating across the client and server tiers:

1. Authentication and Access Control Module

Manages user registration, secure login, password policy enforcement (minimum 8 characters, uppercase, digit, special character), and stateless session handling via JSON Web Tokens (JWT).

Implements Role-Based Access Control (RBAC) across ATHLETE, COACH, and ADMIN security contexts.

2. User & Profile Management Module

Handles personal biographical information, profile photos, sport specialization, physical metrics (height, weight, age), and contact credentials for all registered users.

3. Performance & Drill Tracking Module

Records athletic drills, training volumes, repetition counts, resistance loads, sprint durations, and track times.

Automatically tracks and highlights Personal Records (PRs) and benchmark milestones.

4. AI-Powered Adaptive Workout Generator Module

Integrates Google Gemini Generative AI to dynamically synthesize multi-week, customized workout plans (AiWorkoutPlanDto) based on the athlete's target sport, skill level, and equipment availability.

Breaks down exercise routines with warm-ups, drill mechanics, sets, reps, rest intervals, and recovery protocols.

5. Biometric Wellness & Fatigue Monitoring Module

Captures daily subjective wellness surveys (sleep duration and quality, muscle soreness, hydration, energy, and stress).

Computes real-time athlete readiness scores to proactively identify overtraining and mitigate injury risks.

6. Achievements, Badges & Gamification Module

Facilitates submission of athletic achievement claims accompanied by performance proof (external URLs, competition results, media).

Manages automated badge unlocking (e.g., *First Step Trophy*, *Gold Standard*, *Coach Verified Champion*) upon meeting specific milestone criteria.

7. Coach-Athlete Roster & Connection Module

Manages coach directories, connection requests, and bidirectional coach-athlete roster assignments.

Enables coaches to inspect granular individual athlete dossiers and monitor compliance.

8. Coach Analytics & Automated PDF Reporting Module

Renders real-time multi-dimensional analytics (Radar charts, Line trends) using Recharts.

Generates downloadable, high-resolution official PDF progress reports and athlete evaluation cards using the integrated jsPDF-AutoTable engine.

9. Public Credential & QR Code Verification Module

Generates unique, cryptographic QR Codes linked to each athlete's public verification profile.

Enables unauthenticated external talent scouts, university recruiters, and federations to instantly validate verified milestones at /verify/:id.

10. Direct Communication (In-App Chat) Module

Provides an embedded 1-on-1 messaging channel between athletes and their assigned coaches for immediate feedback, technique review, and training discussions.

11. Real-Time Notification & Alert Module

Dispatches asynchronous alerts for newly assigned training routines, achievement approvals or rejections, unread messages, and scheduled workouts.

12. System Governance & Administrative Audit Logging Module

Delivers complete platform oversight for system administrators, including user account activation/suspension, role elevation, and global roster overrides.

Maintains an immutable audit trail (AdminAuditLog) recording acting administrators, affected entities, action types, IP addresses, and timestamps.

11. Identify the users in your project ?

- Athletes
- Coaches
- Administrator

12. Who owns the system ?

The sports Organization,Athletic Academy/Institution represented by System Administration.

In an operational deployment, AthleteX is owned and governed by the **Athletic Club, Sports Training Institute, University Athletic Department, or Sports Academy** that hosts the platform.

13.Details of the person you have contacted for data collection ?

Organization : Ayras Sports Hub,Nedumkunnam

Name : Abhijith

Place : Nedumkunnam,Kottayam

Contact : 8848108482

Questionnaire to collect details about the project

1.Which sport do you participate in?
Answer: Track and Field (sprints/distance), Football, Basketball, and Strength Athletics.

2. How do you currently maintain your performance records?
Answer: Through manual paper logbooks, phone notes, or shared spreadsheets, which are disorganized and prone to data loss.

3. Do you face difficulties in showcasing your achievements to recruiters?
Answer: Yes, physical certificates and self-reported metrics are difficult to share remotely and are often met with skepticism due to lack of verified proof.

4. Would you prefer maintaining a digital athlete profile?
Answer: Yes, a unified digital profile containing verified stats, badges, and media proof would provide a professional, accessible athletic resume.

5. Which performance statistics are most important for your sport?
Answer: Sprint timing and speed, maximum strength lifts, endurance/stamina levels, and agility ratings.

6. How often do you receive feedback from your coach?
Answer: Inconsistently—mainly through brief verbal discussions during practice, with no permanent written records or historical trend tracking.

7. Would AI-based training recommendations help improve your performance?
Answer: Yes, adaptive AI workouts that adjust sets, reps, and intensity based on daily soreness and fatigue would optimize training and reduce injury risks.

8. What challenges do you face during sports recruitment?
Answer: Limited access to professional scouts, high travel expenses for trials, and the inability to digitally prove that records were achieved under certified supervision.

9. Which features would you like in a sports management platform?
Answer: Performance tracking charts, coach-verified achievement badges, AI workout generator, daily wellness check-ins, and scannable QR verification for scouts.

10. Would you use a mobile application for tracking your sports performance?
Answer: Yes, an accessible mobile-friendly platform for fast workout logging, recovery tracking, and communicating with my coach would be used daily.

3.1.5 GEOTAGGED PHOTOGRAPH



Figure 3.1

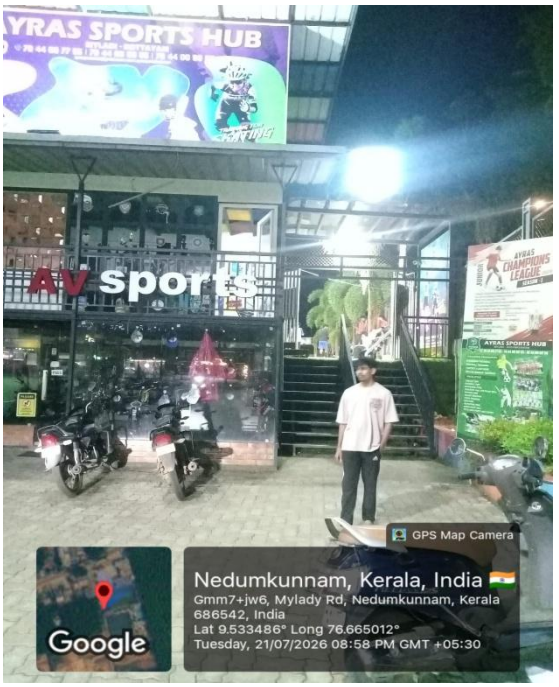


Figure 3.2

3.2 SYSTEM SPECIFICATION

3.2.1 HARDWARE SPECIFICATION

Processor - Intel Core i5 / AMD Ryzen 5 or equivalen

RAM - 8 GB or above

Hard disk - 256 GB SSD or above

3.2.2 SOFTWARE SPECIFICATION

Front End	-React.js, HTML5, CSS3, JavaScript, Tailwind CSS,Recharts,Axios
Back End	-Java21,Springboot3,Spring Web MVC,Spring Data MongoDB,Spring Security,JSON Web Tokens,Apache Maven
Database	- MongoDB
Client on PC	- Windows 10 or above
Technologies used	- React.js, JavaScript, Tailwind CSS,Java21, Spring Boot3 Framework, REST API, MongoDB

3.3 SOFTWARE DESCRIPTION

1. Backend Technologies

- **Java 21 (LTS):** The latest Long-Term Support version of the Java programming language. It serves as the primary programming language for the backend, offering enhanced performance, strong type safety, and modern concurrency features.
- **Spring Boot 3:** An enterprise-grade Java framework used to build production-ready, standalone backend applications. It provides auto-configuration, dependency injection, embedded web server management, and rapid RESTful API development.
- **Spring Web MVC:** The core Spring module used to design and expose RESTful web services. It manages incoming HTTP requests, maps JSON payloads to Java Data Transfer Objects (DTOs), and returns structured JSON responses.
- **Spring Data MongoDB:** A data access abstraction module providing Object-Document Mapping (ODM) and repository interfaces. It enables seamless persistence, querying, and indexing of complex athletic data within MongoDB without boilerplate database code.
- **Spring Security 6:** A comprehensive security framework that enforces authentication and authorization across the application. It secures API endpoints using Role-Based Access Control (RBAC) across ATHLETE, COACH, and ADMIN roles.

- **JSON Web Tokens (JJWT 0.12.6):** A compact, cryptographically signed token utility. It is used to generate and validate stateless session tokens upon user login, ensuring secure, tamper-proof communication between the frontend and backend.
- **Springdoc OpenAPI / Swagger UI 2.6.0:** An automated API documentation engine that inspects Spring controllers to generate interactive Swagger UI documentation, allowing developers to inspect and test endpoints in real time.
- **Apache Maven:** The build automation and dependency management tool used to compile, package, run tests, and manage third-party libraries for the Java backend.

---

2. Frontend Technologies

- **React 19:** A modern, component-based JavaScript library for building dynamic, high-performance Single Page Applications (SPAs). It powers the entire client-side user interface through declarative component states and virtual DOM rendering.
- **Vite:** A next-generation frontend build tool and development server that provides near-instant Hot Module Replacement (HMR) during development and highly optimized production asset bundling.
- **Tailwind CSS v4:** A utility-first CSS framework used to build custom, modern, and fully responsive user interfaces. It enforces a sleek dark-mode aesthetic and consistent visual design across mobile, tablet, and desktop screens.
- **Framer Motion:** A motion and animation library for React used to create smooth micro-interactions, page transition effects, and dynamic UI element reveals.
- **Recharts:** A composable charting library built on React components and SVG. It is utilized to render real-time athletic data visualizations, including multi-axis radar charts (speed, strength, agility) and historical performance line charts.
- **Axios:** A promise-based HTTP client used to send asynchronous API requests from the React frontend to the Spring Boot backend. It includes interceptors to automatically attach JWT authorization headers.

- **React Router DOM v7:** The standard routing library for React applications. It enables seamless client-side page navigation, URL parameter management, and role-based protected routes.
- **React QR Code:** A dedicated React library used to dynamically generate scannable, cryptographic QR codes linking external scouts directly to an athlete's public verified profile (/verify/:id).
- **jsPDF & jsPDF-AutoTable:** Client-side PDF generation libraries used by coaches to compile, format, and export official, high-resolution athlete performance cards and progress summaries.
- **React Hot Toast & React Icons:** Lightweight libraries used to deliver clean, non-intrusive notification banners (toasts) and standardized SVG iconography across all system views.

---

3. Database, AI & DevOps

- **MongoDB (NoSQL Document Database):** A distributed, schema-less document database that stores application data in JSON-like BSON formats. It is ideally suited for storing heterogeneous, high-velocity athletic records, wellness check-in surveys, and audit logs.
- **Google Gemini Generative AI:** Google’s advanced cloud AI model integrated into the backend. It dynamically analyzes an athlete’s sport, target goals, and daily fatigue scores to synthesize customized, multi-week adaptive workout plans.
- **Docker & Docker Compose:** Containerization tools used to package the backend, database, and auxiliary services into lightweight, reproducible containers for seamless local development and multi-environment deployment.

---

4. Testing & Quality Assurance Technologies

- **JUnit 5:** The modern unit testing framework for the Java platform, utilized to write and execute unit tests for core backend business logic, validation rules, and algorithms.
- **Mockito:** A Java mocking framework used in conjunction with JUnit to mock repository layers and external service dependencies, ensuring isolated testing of service-layer logic.
- **Spring MockMvc:** A specialized Spring testing framework used to test REST controllers and HTTP request/response mappings without deploying the application to a live web server.
- **Vitest & React Testing Library:** A fast, Vite-native unit testing framework paired with React Testing Library to validate frontend component rendering, user interactions, and route protection logic in a simulated browser DOM (JSDOM).
- **Selenium WebDriver 4 & ChromeDriver:** An industry-standard browser automation framework used to perform automated End-to-End (E2E) testing on Chrome, asserting full user journeys (homepage navigation, login authentication, and UI element rendering).

CHAPTER 4

SYSTEM DESIGN

4.1 INTRODUCTION

The system design phase is the critical transitional stage in the software engineering lifecycle where abstract functional requirements and domain specifications are converted into a concrete, technical blueprint. It establishes both the high-level architectural framework and the low-level structural components necessary to ensure system scalability, maintainability, data integrity, and security. During this phase, critical engineering decisions are finalized—encompassing overall system topology, client-server interaction patterns, database schema structures, interface design, module decomposition, and security boundaries. By defining how individual software entities interact and process data flows, the design phase ensures that the resulting application fulfills its functional objectives while adhering to strict non-functional benchmarks such as high availability, fault tolerance, and low response latency.

In the context of the **AthleteX** platform, the system design follows a decoupled, three-tier micro-monolithic architectural model designed to handle heterogeneous athletic telemetry and multi-stakeholder workflows. The presentation tier is engineered as an ultra-responsive, component-driven Single Page Application (SPA) using React 19 and Tailwind CSS, communicating asynchronously via RESTful endpoints with the enterprise business logic tier powered by Java 21 and Spring Boot 3. The persistence tier utilizes MongoDB's flexible, document-oriented NoSQL model to manage high-velocity drill logs, wellness surveys, and audit histories without the friction of rigid relational constraints. Furthermore, the design explicitly incorporates stateless JSON Web Token (JWT) security filters with fine-grained Role-Based Access Control (RBAC), external Generative AI service integration for dynamic conditioning synthesis, and cryptographic QR code generation algorithms for public credential verification, creating an end-to-end, resilient blueprint for modern athletic development.

4.2 UML DIAGRAM

UML stands for Unified Modelling Language, which is a visual language used to model software systems. UML diagrams are graphical representations of software systems that provide a standard notation for developers to communicate and understand software designs. UML diagrams can be used to model different aspects of software systems, including use cases, class structures, sequence diagrams, activity diagrams, and more. Each type of diagram represents a different aspect of the system, and they can be used to document, analyses, design, and communicate software systems. UML diagrams include

- Use Case Diagram
- Sequence Diagram
- State Chart Diagram
- Activity Diagram
- Class Diagram
- Object Diagram
- Component Diagram
- Deployment Diagram
- Collaboration Diagram

4.2 UML DIAGRAM

A standardized dialect called Unified Modeling Language (UML) is utilized to conceptualize, characterize, plan, and depict program frameworks. The Question Administration Gather (OMG) was dependable for creating UML, and the primary draft of the UML 1.0 definition was discharged in January 1997. Programming dialects like Java, C++, and COBOL are not the same as UML. It could be a nonexclusive visual demonstrating dialect utilized for computer program frameworks and a pictorial dialect utilized for program outlines. UML may be utilized for nonsoftware frameworks, such as fabricating forms, indeed though it is generally utilized to speak to program frameworks.

• Class diagram

• Object diagram

• Use case diagram

• Sequence diagram

• Collaboration diagram

• Activity diagram

• State chart diagram

• Deployment diagram

• Component diagram

4.2.1 Use Case Diagram

A use case diagram is a graphical representation that illustrates how users and external entities interact with a system. It helps identify, outline, and organize the system's functional requirements from a user's perspective. Built using the Unified Modeling Language (UML), use case diagrams define system boundaries, actors (users or external systems), use cases (specific actions or functionalities), and their relationships. They are essential for understanding system behavior, verifying requirements, testing, and providing customer support. To create an effective use case diagram, it is important to use clear naming conventions, accurately depict actor roles, highlight essential functionalities, and avoid unnecessary complexity. Following these principles ensures a well-structured diagram that effectively communicates the system's core functionalities.

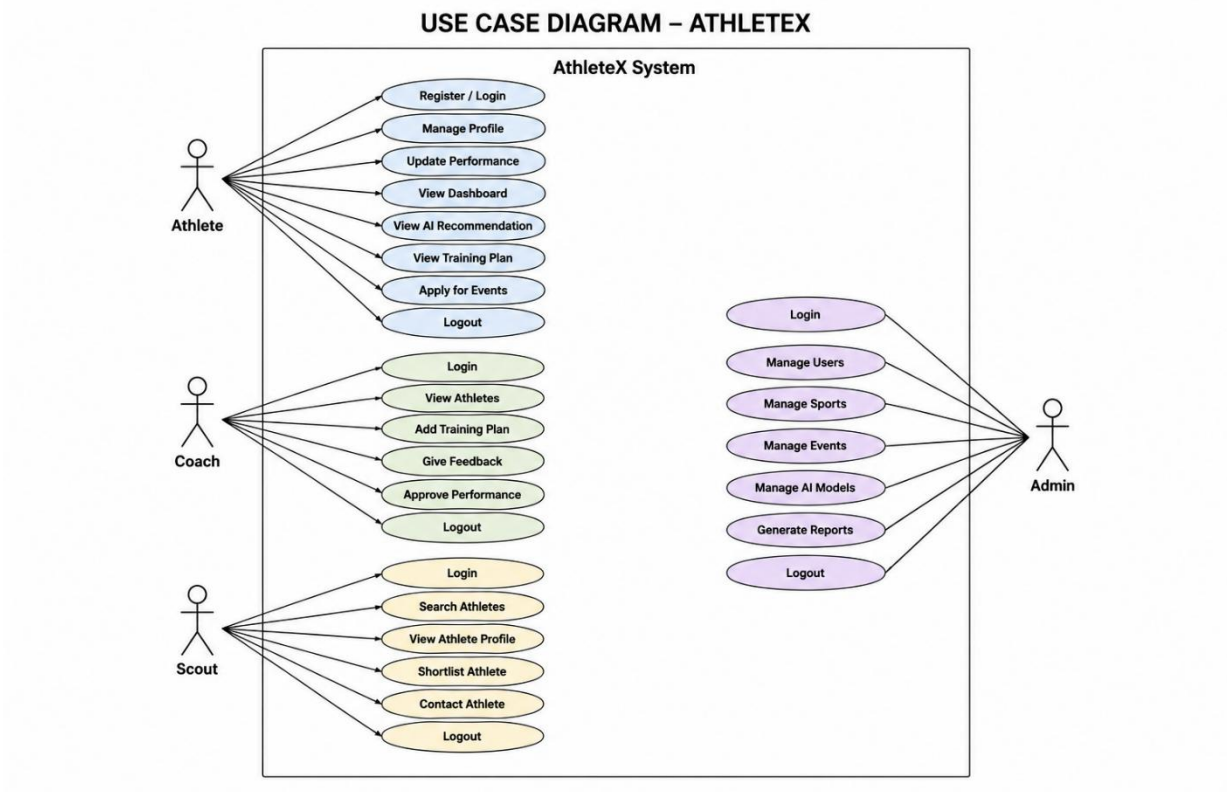


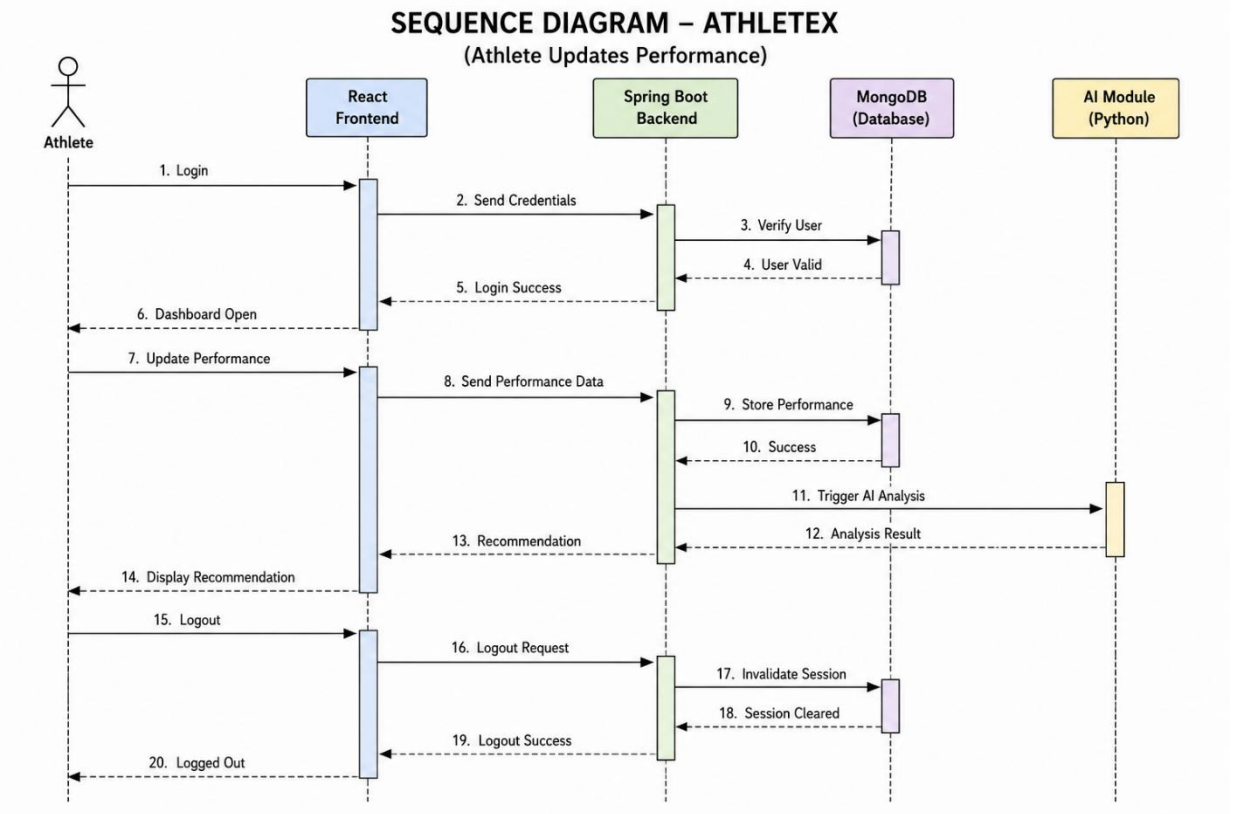
Figure 1 : Use case diagram for Freematch AI

4.2.2 Sequence Diagram

A sequence diagram is a type of interaction diagram that depicts the chronological flow of messages between system components. It helps visualize object interactions, understand system requirements, and detect potential issues. Key elements include actors (external users or systems), lifelines (vertical dashed lines representing an object's lifespan), messages (interactions between objects), and guards (conditions that control message flow). Sequence diagrams are used to model complex processes, illustrate use case details, and analyze system functionality. They aid both developers and business professionals in understanding and refining system behavior effectively

Uses of sequence diagram

- Modeling and visualizing the logic of complex functions, operations, or procedures.
- Showing details of UML use case diagrams.
- Understanding the detailed functionality of current or future systems.
- Visualizing how messages and tasks move between objects or components in a system.
- Overall, sequence diagrams are useful for representing the flow of interactions.



4.2.3 State Chart Diagram

A State Chart Diagram is a UML behavioral diagram that represents the various states an object or system can be in and how it transitions between them. It consists of key elements like the Initial State (starting point), State (current condition of an object), Transition (movement between states), Event and Action (triggers and responses), Signal (messages causing transitions), and the Final State (end of the process). State diagrams help visualize system behavior over time, making them useful for modeling dynamic systems and workflow processes.

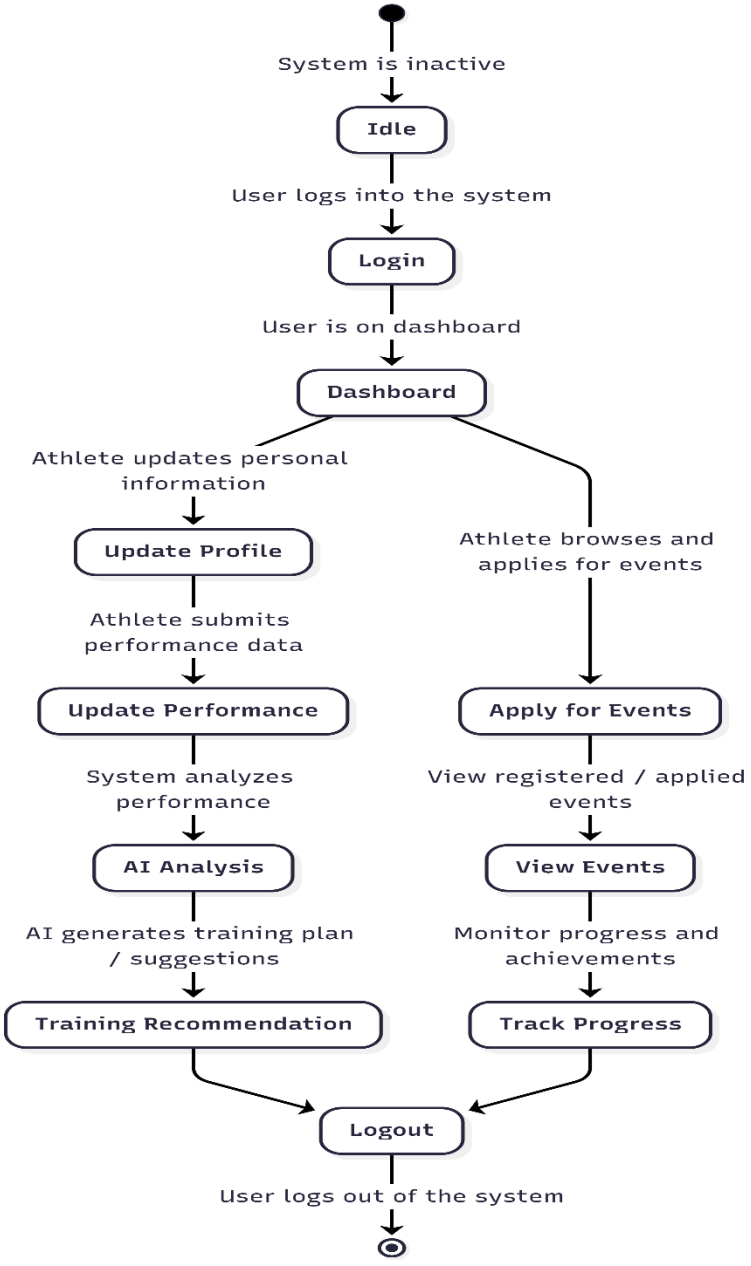


Figure 3 : State Chart diagram for AthleteX

4.2.4 Activity Diagram

An activity diagram is a visual representation of a workflow that shows how one activity leads to another. An activity is referred to as a system operation, and one operation leads to another in the control flow. A flow can be parallel, concurrent, or branched, and activity diagrams use various functions such as branching, joining, etc., to manage all types of flow control. Activity diagrams are a type of behavior diagram that shows the behavior of a system. They show the flow of control from the start point to the end point and show the different decision paths that exist during the execution of the activity.

The key components of an activity diagram are:

- **Initial node**: A starting point of the activity diagram, denoted by a black circle.
- **Activity**: A task or action performed by the system or entity, represented by a rectangle with rounded corners.
- **Control flow**: It represents the sequence of activities or actions performed by the system or entity, represented by an arrow.
- **Decision node**: A decision or branching point in the activity flow, denoted by a diamond shape.
- **Merge node**: Used to merge multiple branches of the activity flow into a single flow, represented by a diamond shape with a plus sign inside.
- **Fork node**: Used to split the activity flow into multiple parallel flows, represented by a solid black circle with multiple arrows.
- **Join node**: Used to join multiple parallel flows back into a single flow, represented by a solid black circle with multiple arrows pointing towards it.
- **Final node**: The end point of the activity diagram, denoted by a black circle with a dot inside.
- **Object flow**: Represents the flow of objects or data between activities, represented by a dashed arrow.

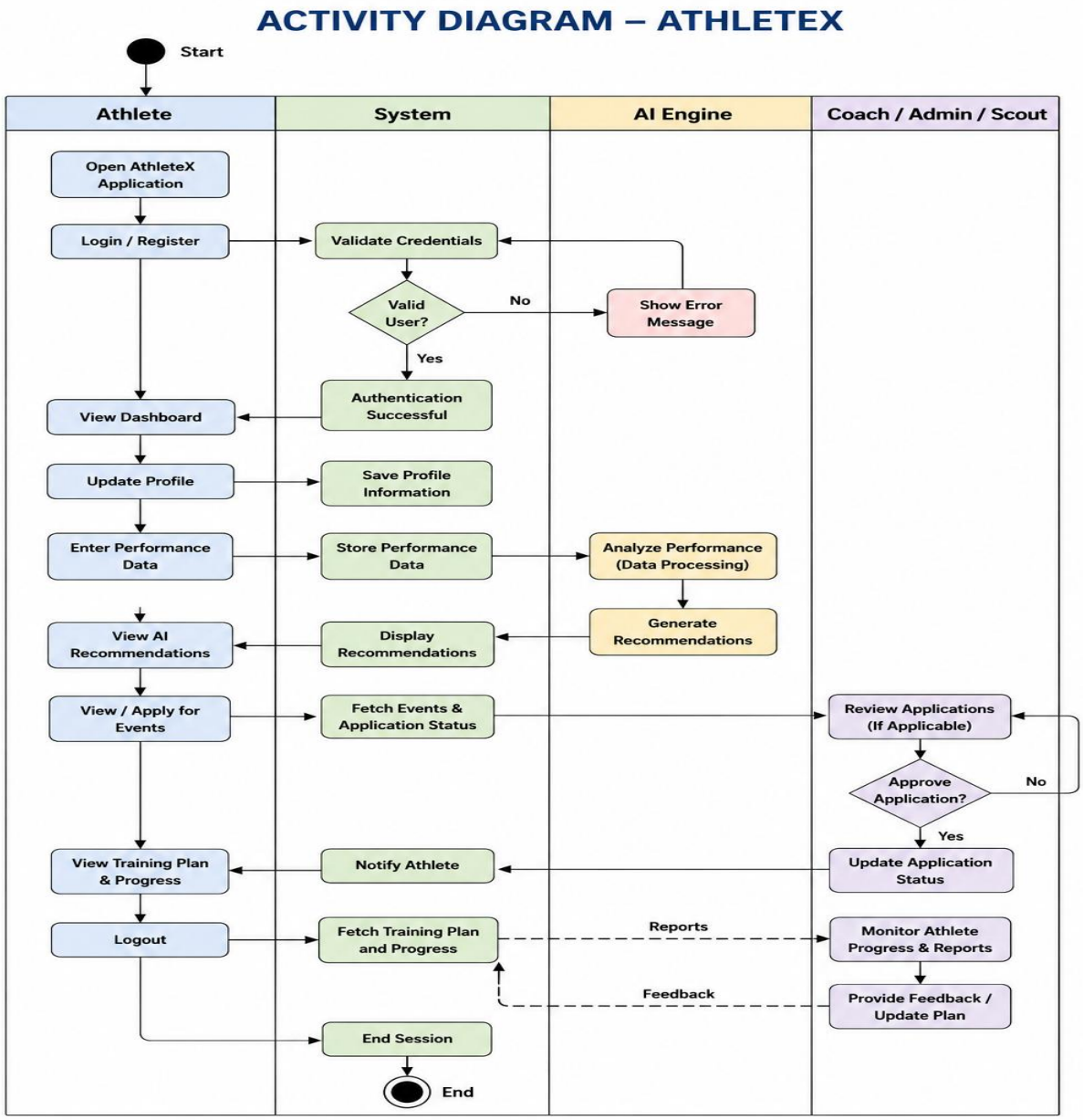


Figure 4 : Activity diagram for AthleteX

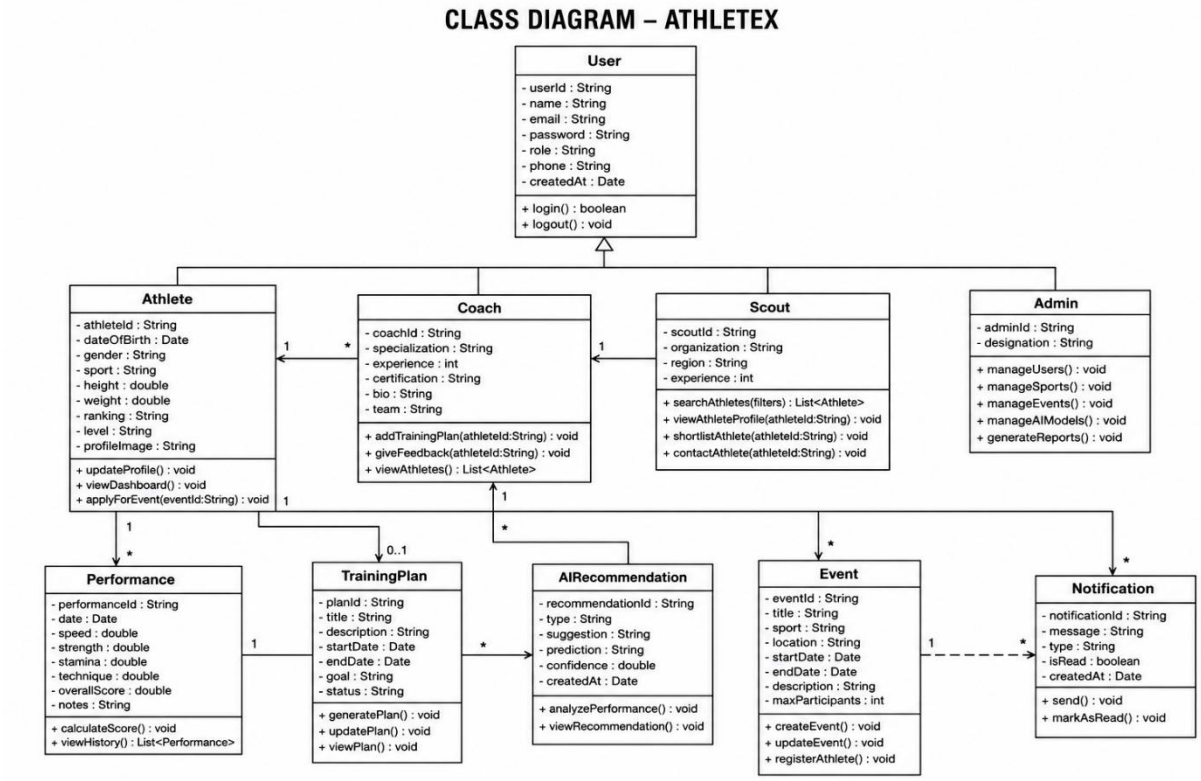
4.2.5 Class Diagram

A Class Diagram is a key UML component used in object-oriented modeling to represent the structure of an application. It defines classes, objects, interfaces, and relationships such as association, aggregation, composition, inheritance, dependency, and multiplicity. Classes are depicted as rectangles containing attributes and methods, while interfaces define abstract methods. Relationships like aggregation (whole-part), composition (stronger whole-part), and inheritance (is-a hierarchy) establish connections between classes. Class diagrams provide a

conceptual and detailed view of system structure, making them essential for both software development and data modeling.

Important Components of a Class Diagram:

- **Class:** A blueprint for objects, represented as a rectangle with a class name, attributes, and methods.
- **Interface:** Defines abstract methods, represented as a circle with the interface name.
- **Object:** An instance of a class with state and behavior, shown as a rectangle.
- **Association:** A link between two classes, depicted as a line with optional direction and multiplicity.
- **Aggregation:** A part-whole relationship, shown with a hollow diamond at the whole side.
- **Composition:** A stronger part-whole relationship, represented with a filled diamond.
- **Inheritance:** An "is-a" relationship between superclass and subclass, marked by an open arrow.
- **Dependency:** One class depends on another, shown with a dashed arrow.



4.2.6 Object Diagram

Class diagrams and object diagrams are closely related in object-oriented modeling. Object diagrams are instances of class diagrams, which represent a snapshot of the system at a given moment in time. Both types of diagrams use the same concepts and notation to represent the structure of a system. While class diagrams are used to model the structure of the system, including its classes, attributes, and methods, object diagrams represent a group of objects and their connections at a specific point in time. An object diagram is a type of structural diagram in UML that shows instances of classes and their relationships.

Key Components of an Object Diagram:

- **Object:** An instance of a class representing a specific entity, shown as a rectangle.
- **Class:** A template for objects with attributes and methods, represented as a three-part rectangle.
- **Link:** A relationship between objects, depicted as a line.
- **Attribute:** A property defining an object's state, shown as a name-value pair.
- **Value:** A specific instance of an attribute.
- **Operation:** An action an object can perform, listed inside the class rectangle.
- **Multiplicity:** Defines the number of instances in a relationship.

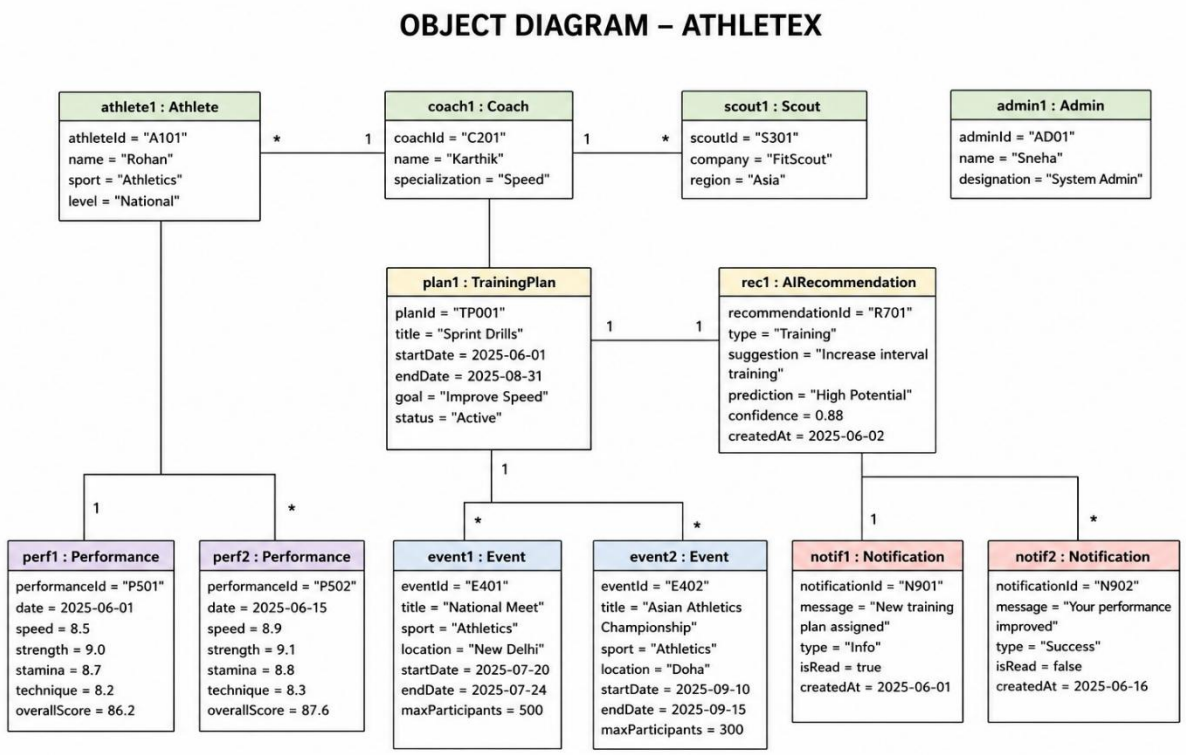


Figure 6 : Object diagram for AthleteX

4.2.7 Component Diagram

A component diagram in UML illustrates how various components are interconnected to create larger components or software systems. It is an effective tool for representing the structure of complex systems with multiple components. By using component diagrams, developers can easily visualize the internal structure of a software system and understand how different components work together to accomplish a specific task.

Key Components of a Component Diagram:

- **Component**: A modular unit with defined functionality, shown as a rectangle.
- **Interface**: A set of accessible methods, represented as a circle.
- **Port**: A connection point for interactions, shown as a small square.
- **Connector**: A link enabling communication, depicted as a line.
- **Dependency**: A relationship where one component relies on another, shown as a dashed arrow.

- **Association**: A connection between components, represented as a line.

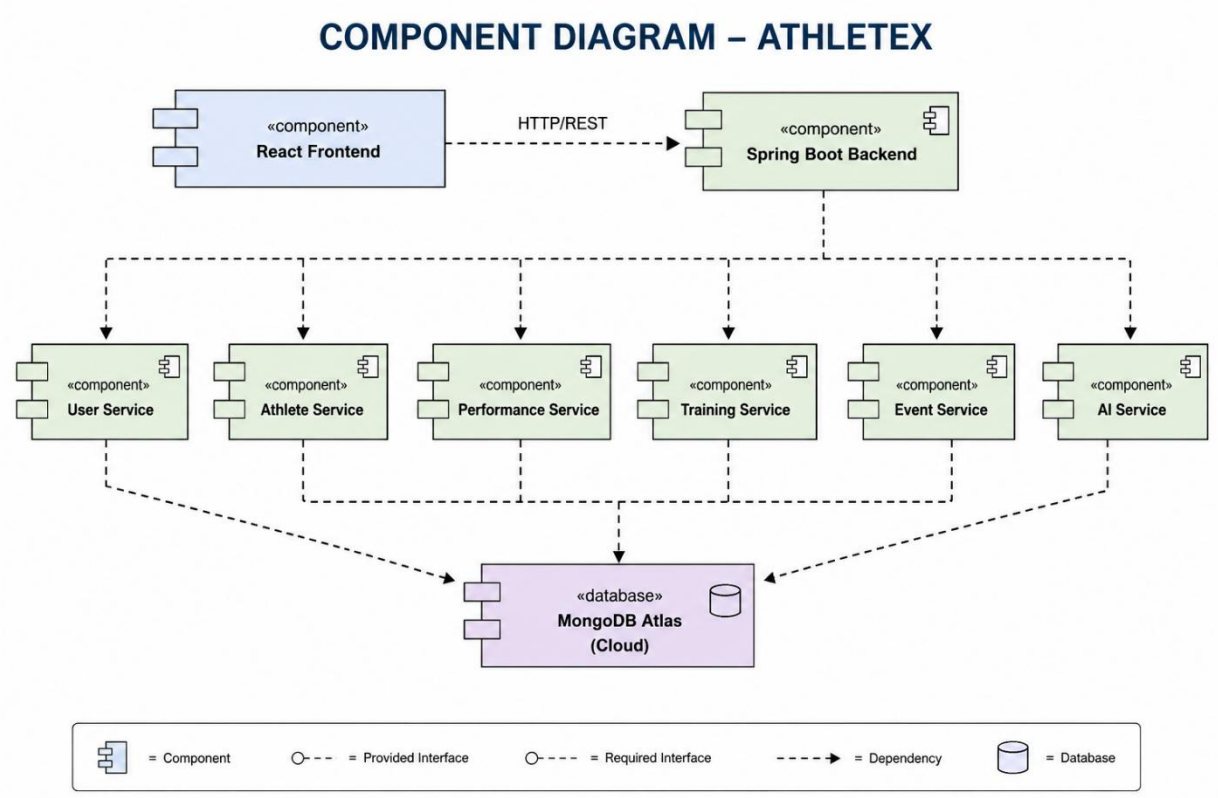


Figure 7 : Component diagram for AthleteX

4.2.8 Deployment Diagram

A deployment diagram is a type of UML diagram that focuses on the physical hardware used to deploy software. It provides a static view of a system's deployment and involves nodes and their relationships. The deployment diagram maps the software architecture to the physical system architecture, showing how the software will be executed on nodes. Communication paths are used to illustrate the relationships between the nodes. Unlike other UML diagram types, which focus on the logical components of a system, the deployment diagram emphasizes the hardware topology.

Key Components of a Deployment Diagram:

- **Node**: A physical or virtual machine hosting components, shown as a box.
- **Component**: A software element providing functionality, represented as a rectangle.
- **Artifact**: A physical data piece used or produced by a component.
- **Deployment Specification**: Details how a component or artifact is deployed, including location and configuration.

- **Association**: A relationship between nodes and components, shown as a connecting line.
- **Communication Path**: A network or communication link between nodes, represented by a labeled line.

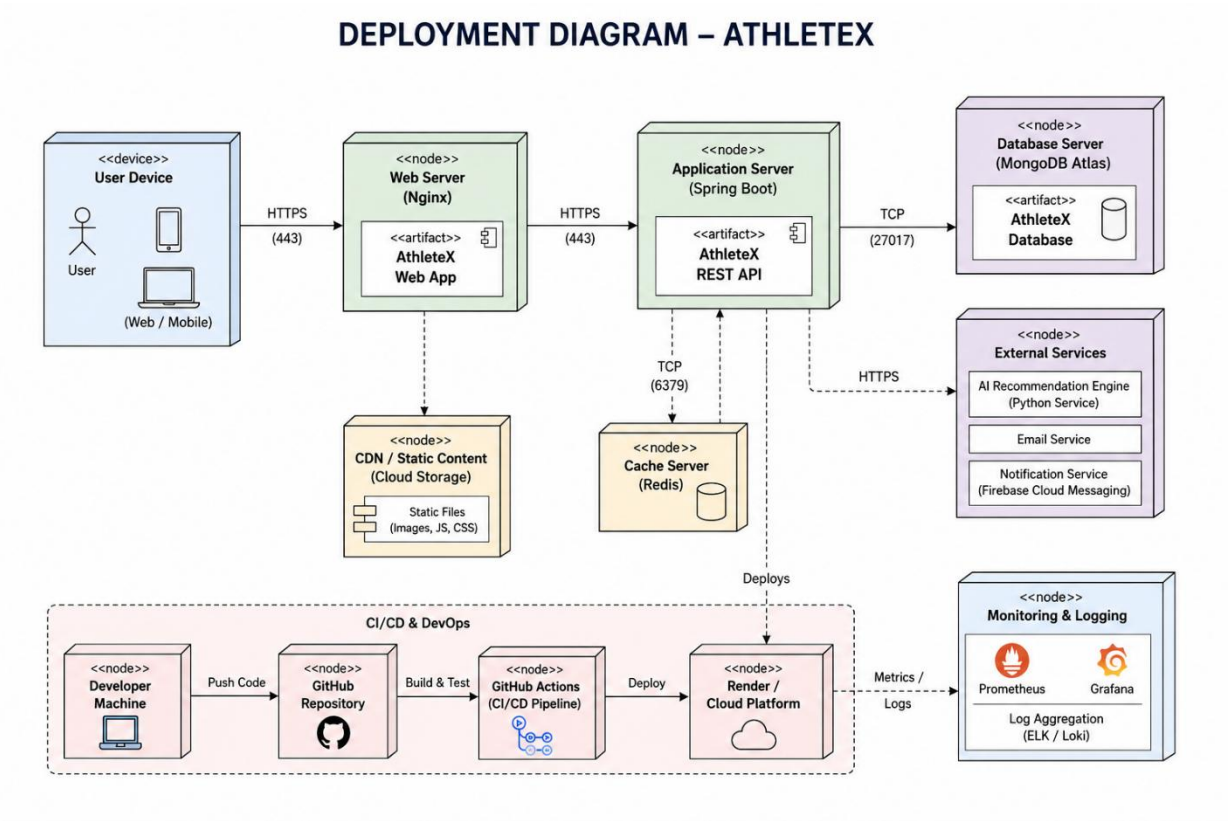


Figure 8 : Deployment diagram for AthleteX

4.3 USER INTERFACE DESIGN USING FIGMA

Form Name: Landing Page

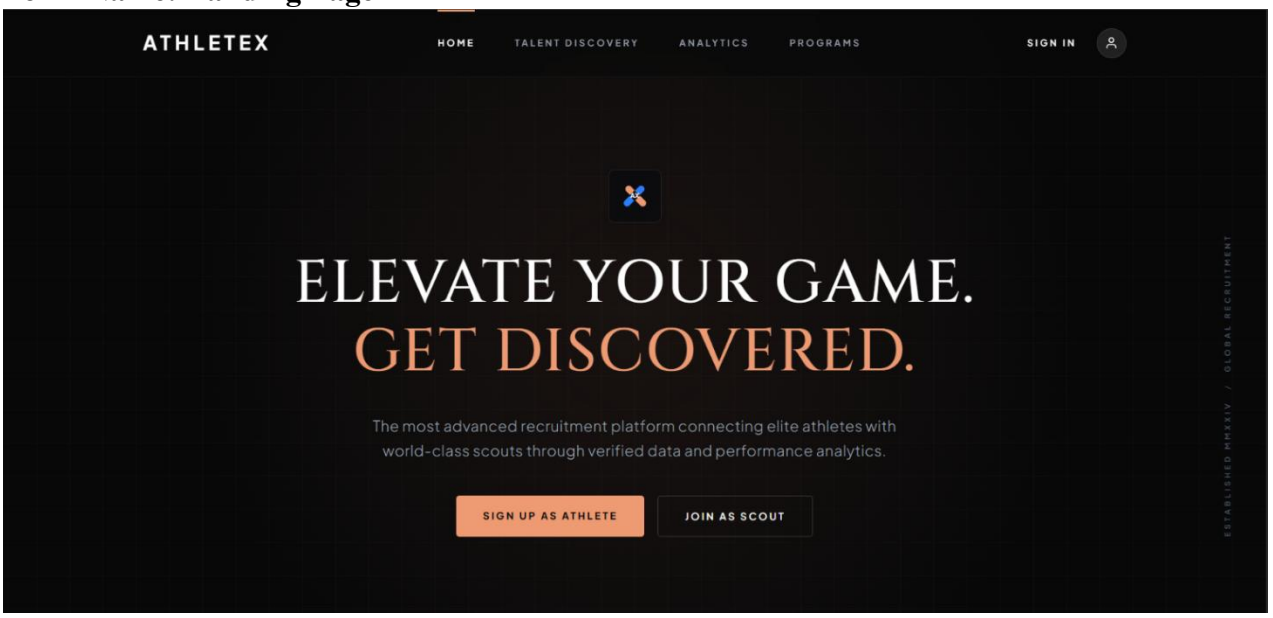


Figure : Landing Page

Form Name: Login Page

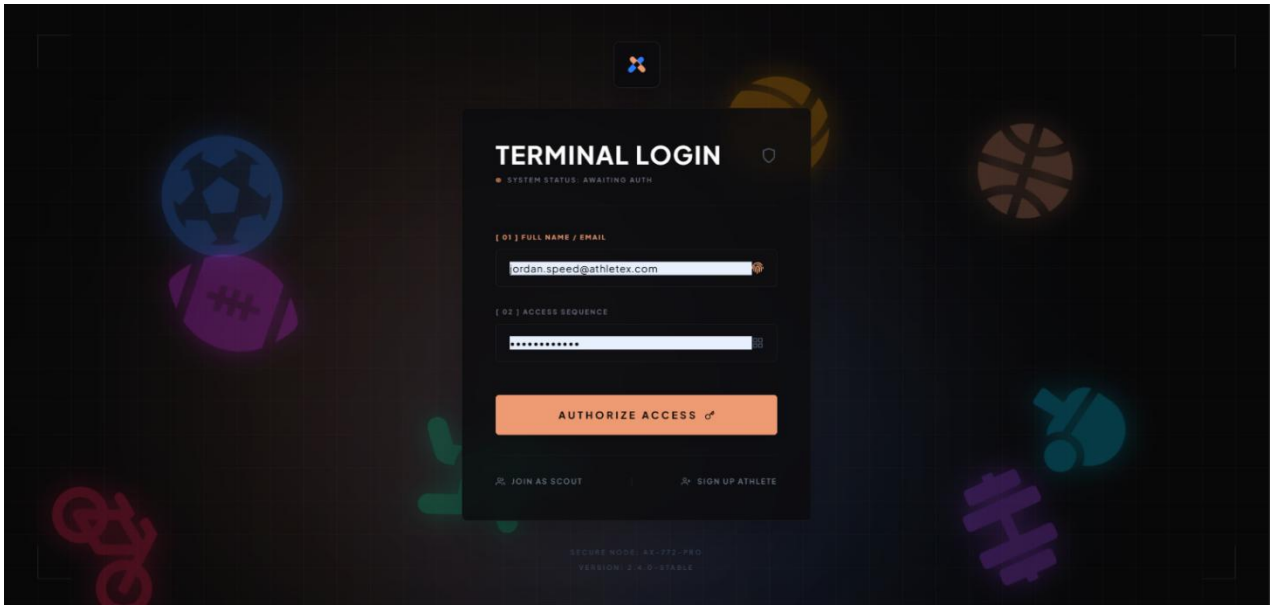


Figure : Login Page

Form Name: Sign Up Page

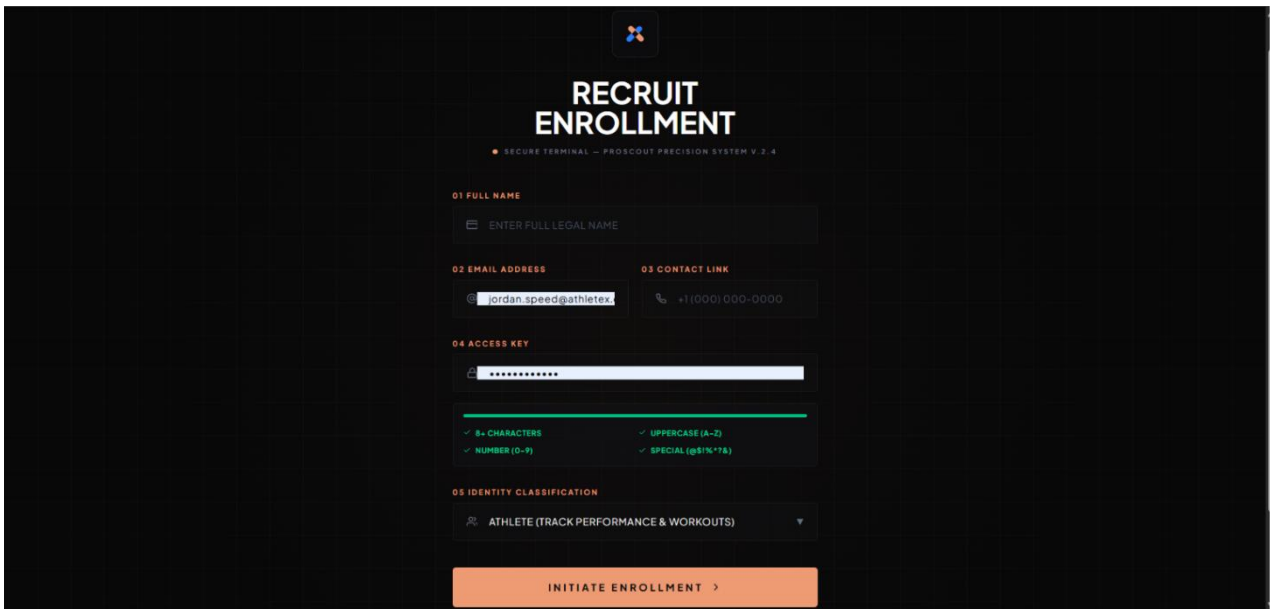


Figure : User Regsitration Page

Form Name: Athlete Dashboard

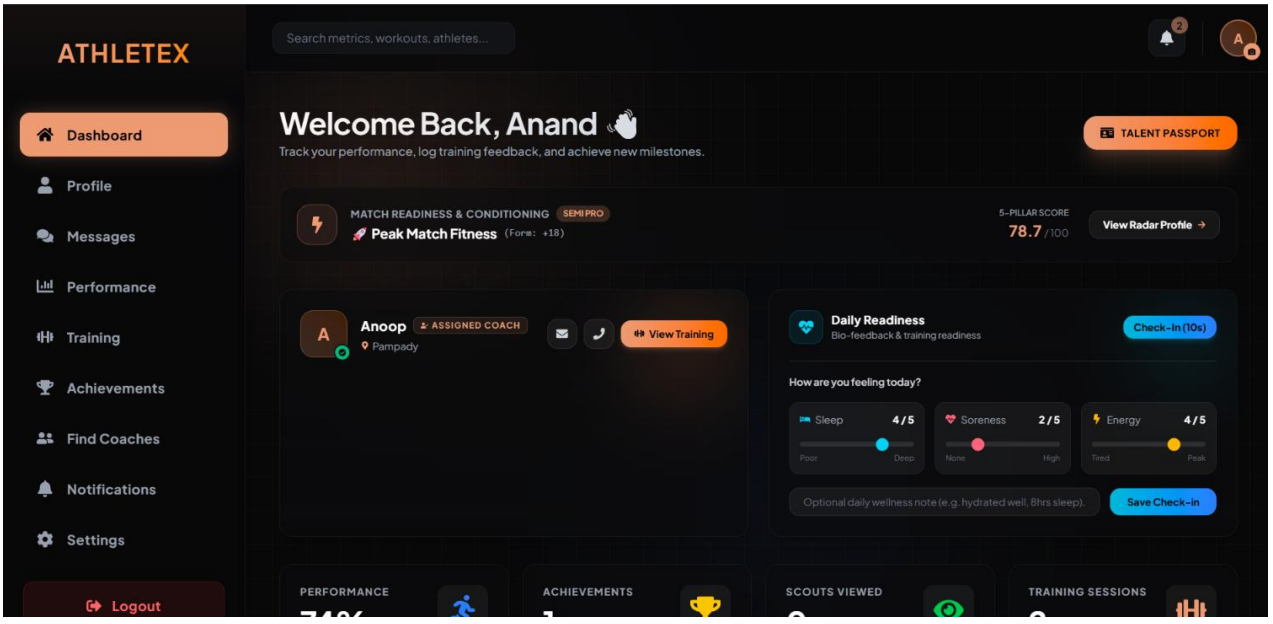


Figure : Athlete Dashboard Page

Form Name: Coach Dashboard

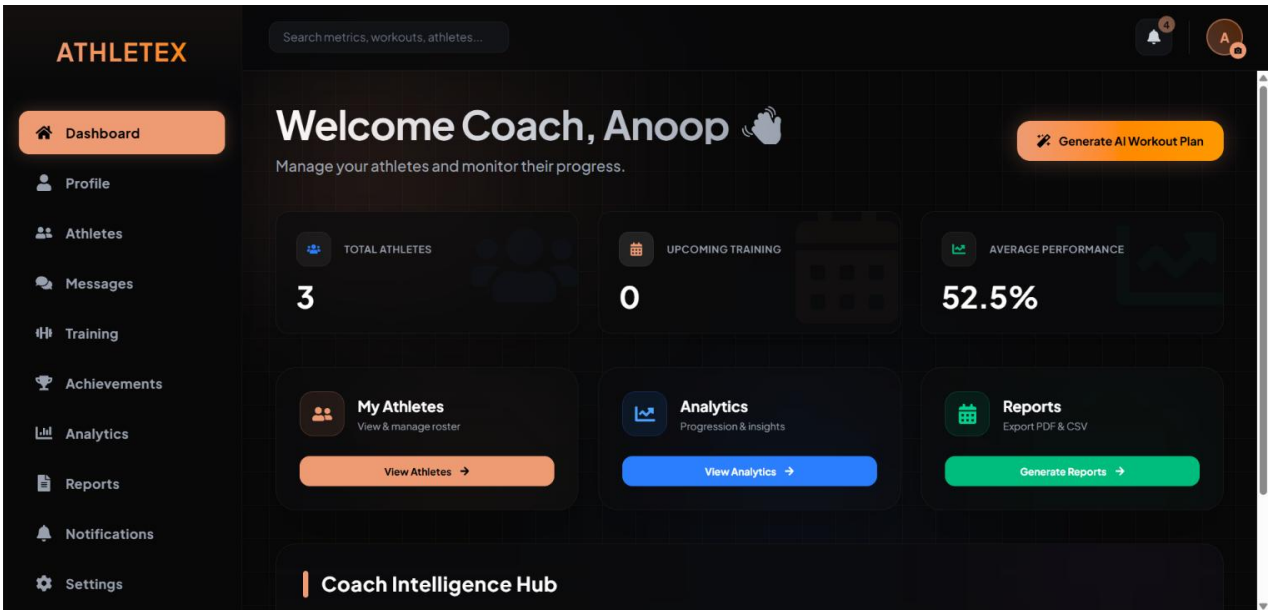


Figure : Coach Dashboard

Form Name: Admin Dashboard

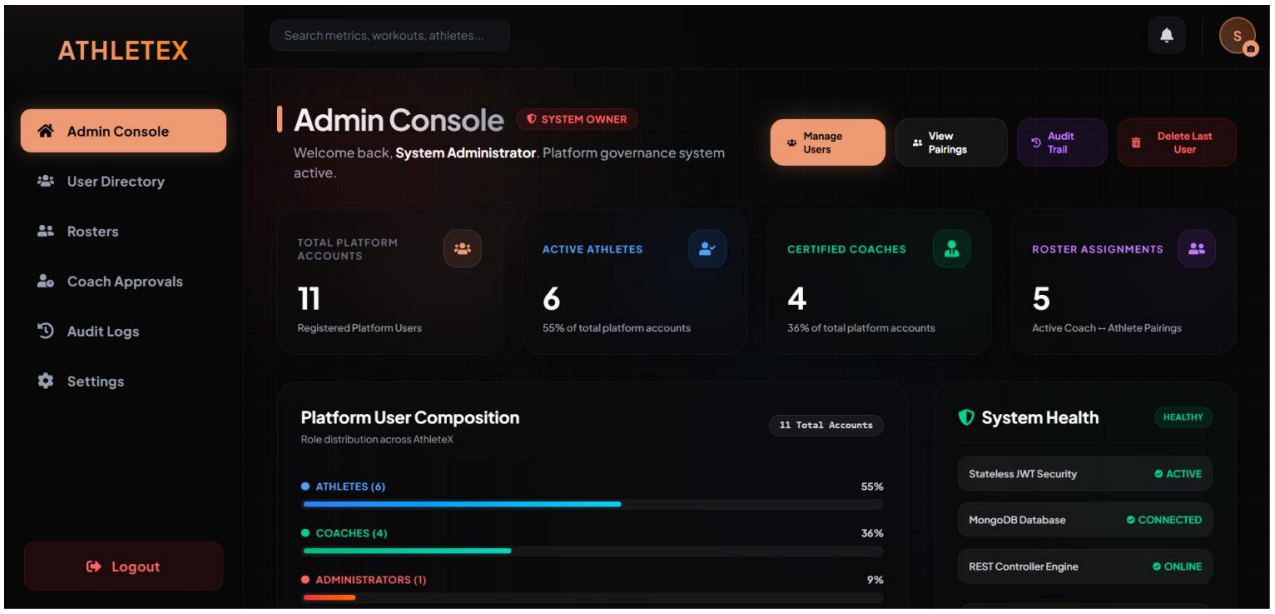


Figure :Admin Dashboard

4.4DATABASE DESIGN

4.4.1. Introduction to Database Design

The database design phase establishes the logical and physical data persistence architecture for the AthleteX platform. Athletic performance tracking inherently deals with dynamic, heterogeneous, and time-series data—including continuous workout repetitions, variable drill parameters (sprints, weights, shuttle runs), subjective biometric wellness surveys, and multimedia credential proofs.

To satisfy these data requirements, AthleteX utilizes MongoDB, an enterprise-grade, distributed NoSQL document database, interfaced via Spring Data MongoDB. Unlike traditional Relational Database Management Systems (RDBMS) that enforce rigid table schemas and computationally expensive multi-table joins, MongoDB's JSON/BSON document model offers:

Dynamic Schema Flexibility: Enables drill entries and achievements to encapsulate varying attributes (e.g., sprint distance vs. bench press weight vs. vertical jump height) without database schema migrations.

High-Throughput Ingestion: Seamlessly handles rapid, concurrent writes of daily wellness check-ins, real-time chat messages, and live drill telemetry.

Natural Aggregation & Querying: Facilitates instantaneous calculation of personal records (PRs), squad-level radar metrics, and longitudinal performance trends.

4.4.2 Schema Design Strategy

The schema design strategy for **AthleteX** employs a **hybrid referencing-and-embedding architecture** tailored to balance read-heavy dashboard performance with high-velocity athletic telemetry writes. Rather than attempting to nest unbounded arrays within a single monolithic user document—which would risk exceeding MongoDB's 16MB BSON document limit and degrade write throughput—the architecture decouples high-frequency datasets (such as performance_drills, wellness_checkins, and chat_messages) into dedicated, independent collections linked via logical foreign keys (userId, coachId, athleteId). To optimize query latency and eliminate computationally expensive multi-collection $lookup aggregations, the design selectively employs **targeted denormalization** of critical display metadata, such as senderName in chat records and verifiedByCoachName in achievement documents. This approach is reinforced by strategic **compound indexing** on time-series and query-intensive paths—including { conversationId: 1, timestamp: -1 } for instant message retrieval and { userId: 1, date: 1 } to prevent

duplicate daily check-ins—ensuring strong data integrity, zero join overhead, and millisecond-level response times across all client dashboards.

4.4.3 Logical Data Architecture & Entity Relationships

The data model for AthleteX is structured around a hybrid referencing-and-embedding strategy. Core entities are stored in dedicated collections with standardized Object IDs (id), while relational associations are maintained through logical foreign keys (userId, athleteId, coachId) to preserve document boundaries and avoid unbound array growth.

Entity Relationship Summary

User (users): The primary entity representing system actors. A single user can act as an ATHLETE, COACH, or ADMIN.

Performance Drills (performance_drills): An Athlete has a 1-to-Many (1:N) relationship with Performance Drills. Each drill captures quantifiable metrics (duration, weight, repetitions) along with coach verification metadata.

Achievements (achievements): An Athlete has a 1-to-Many (1:N) relationship with Achievements. Each record holds proof links, verification statuses, and coach accreditation.

Wellness Check-Ins (wellness_checkins): An Athlete has a 1-to-Many (1:N) relationship with daily biometric logs (sleep quality, muscle soreness, energy levels, and readiness score).

Coach-Athlete Assignments (coach_athlete_assignments): Maintains a Many-to-Many (M:N) associative bridge linking Coaches to Athletes with status flags (ACTIVE, PENDING, TERMINATED).

Trainings (trainings): A Coach creates custom workout assignments linked 1-to-Many (1:N) to specific Athletes with perceived exertion ratings (RPE).

Chat Messages (chat_messages): Users participate in conversations linked by a composite conversationId (senderId_recipientId), sorted chronologically.

Notifications (notifications): A User has a 1-to-Many (1:N) relationship with alert notifications.

Admin Audit Logs (admin_audit_logs): Maintains an immutable audit log of administrative actions, recording acting admin IDs, target entity IDs, timestamps, and action types.

4.4.4 Indexing, Performance Optimization & Integrity

Compound Indexing for Low-Latency Lookups:

Chat Messages: Compound index on { conversationId: 1, timestamp: -1 } to fetch chronological chat threads within milliseconds.

Performance Drills: Compound index on { userId: 1, drillCode: 1, value: 1 } to evaluate and update Personal Records (PRs) instantaneously.

Wellness Telemetry: Compound unique index on { userId: 1, date: 1 } preventing duplicate daily submissions while accelerating calendar queries.

Denormalization for Query Efficiency:

Critical display fields such as senderName, verifiedByCoachName, and targetEntityName are selectively denormalized into child documents. This eliminates the necessity of multi-document lookup queries during high-concurrency read operations.

Stateless Security Consistency:

User documents do not store transient JWT session states. The database stores only the immutable identity, role, and hashed credentials, keeping database I/O minimal during authenticated API interactions.

4.4 TABLE DESIGN

Table 1: Users Collection (users)

Description: Stores account credentials, role classifications, athlete physical profiles, and coach accreditations.

Field Name	Data Type	Key / Index	Null / Constraint	Description
id	String (ObjectId)	Primary Key (@Id)	Not Null, Unique	Unique identifier for the user account.
fullName	String	None	Not Null	Full legal name of the user.
email	String	Unique Index	Not Null, Valid Email	Unique email address used for login authentication.
phone	String	None	Not Null	Primary contact phone number.

Field Name	Data Type	Key / Index	Null / Constraint	Description
password	String	None	Not Null, BCrypt Encrypted	Securely hashed password.
role	String (Enum)	Secondary Index	Not Null (ATHLETE, COACH, ADMIN)	Access level defining authorized system features.
otp	String	None	Nullable	One-time password token for account verification.
isVerified	Boolean	None	Default: false	Status indicating if email/phone has been verified.
sport	String	None	Nullable (Athlete)	Primary sport discipline (e.g., Football, Track, Basketball).
position	String	None	Nullable (Athlete)	Specific playing position or role (e.g., Sprinter, Striker).
age	Integer	None	Nullable (Athlete)	Age of the athlete in years.
gender	String	None	Nullable (Athlete)	Gender category.
height	Double	None	Nullable (Athlete)	Height in centimeters (cm).
weight	Double	None	Nullable (Athlete)	Body mass in kilograms (kg).
city	String	None	Nullable	City of residence/training.
state	String	None	Nullable	State / Province.
country	String	None	Nullable	Country of residence.
bio	String	None	Nullable	Brief athletic biography or coaching statement.
profileImage	String	None	Nullable	URL or storage path to avatar picture.
specialization	String	None	Nullable (Coach)	Coaching specialty (e.g., Strength & Conditioning).

Field Name	Data Type	Key / Index	Null / Constraint	Description
experienceYears	Integer	None	Nullable (Coach)	Total coaching experience in years.
certifications	String	None	Nullable (Coach)	Accredited licensing credentials.
title	String	None	Nullable (Coach)	Official professional title (e.g., Head Sprint Coach).
verificationStatus	String (Enum)	None	Default: APPROVED	Coach accreditation status (APPROVED, PENDING, REJECTED).

Table 2: Performance Drills Collection (performance_drills)
Description**:** Records quantitative athletic drill results, sprint times, resistance loads, and coach verification data.

Field Name	Data Type	Key / Index	Null / Constraint	Description
id	String (ObjectId)	Primary Key (@Id)	Not Null, Unique	Unique drill log identifier.
userId	String	Foreign Key (users.id)	Not Null, Indexed	ID of the athlete who performed the drill.
drillCode	String	Compound Index	Not Null	Standardized code (e.g., 40_YARD_DASH, BENCH_PRESS_1RM).
drillName	String	None	Not Null	Descriptive drill title (e.g., "40-Yard Dash").
category	String	Secondary Index	Not Null (SPEED, STRENGTH, ENDURANCE, AGILITY)	Fitness attribute category.
value	Double	None	Not Null	Numeric performance score achieved (e.g., 4.45).
unit	String	None	Not Null	Measurement unit (s, kg, lbs, cm, min).

Field Name	Data Type	Key / Index	Null / Constraint	Description
lowerIsBetter	Boolean	None	Not Null	true for speed/timed drills; false for weight/height.
isPersonalRecord	Boolean	None	Default: false	Flag indicating if this entry is the athlete's all-time best.
percentile	Double	None	Nullable (0.0 – 100.0)	Benchmark percentile rank relative to standardized cohorts.
tierBadge	String	None	Nullable	Attained performance tier (Elite, Pro, Varsity).
notes	String	None	Nullable	Athlete remarks regarding weather, footwear, or conditions.
proofUrl	String	None	Nullable	Video link or third-party telemetry URL (YouTube/Strava).
recordedAt	LocalDateTime	Compound Index	Not Null	Date and time the drill was executed.
isCoachVerified	Boolean	None	Default: false	Whether certified coach reviewed and approved this score.
verificationStatus	String (Enum)	None	PENDING, APPROVED, REJECTED	Official status of the drill verification.
coachFeedback	String	None	Nullable	Critique and guidance from verifying coach.
verifiedAt	LocalDateTime	None	Nullable	Timestamp when coach verified the drill.
verifiedByCoachId	String	Foreign Key (users.id)	Nullable	Identifier of verifying coach.
verifiedByCoachName	String	None	Nullable	Denormalized coach name for quick rendering.

Table 3: Achievements Collection (achievements)

Description: Manages athlete competition victories, certificates, and verified accolades.

Field Name	Data Type	Key / Index	Null / Constraint	Description
id	String (ObjectId)	Primary Key (@Id)	Not Null, Unique	Unique achievement identifier.
athleteId	String	Foreign Key (users.id)	Not Null, Indexed	Identifier of the athlete claiming the milestone.
coachId	String	Foreign Key (users.id)	Nullable	Coach under whose supervision the milestone occurred.
title	String	None	Not Null	Headline of the award (e.g., "State Track Gold Medal").
description	String	None	Not Null	Narrative description of tournament and accomplishment.
category	String	None	Not Null	Milestone classification (e.g., Tournament, Fitness, Academic).
level	String	None	Not Null	Competition tier (Club, Regional, State, National).
date	LocalDate	None	Not Null	Date the achievement was awarded.
icon	String	None	Nullable	Visual badge or icon identifier.
proofUrl	String	None	Nullable	URL to certificate image, official

Field Name	Data Type	Key / Index	Null / Constraint	Description
				score sheet, or media.
proofType	String	None	IMAGE, DOCUMENT, LINK	Format classification of attached evidence.
isVerified	Boolean	Secondary Index	Default: false	Digital verification indicator.
verifiedByCoachId	String	Foreign Key (users.id)	Nullable	ID of validating coach.
verifiedByCoachName	String	None	Nullable	Denormalized validating coach display name.
verifiedAt	LocalDateTime	None	Nullable	Timestamp of coach accreditation.
createdAt	LocalDateTime	None	Auto-generated	Timestamp when achievement claim was submitted.
updatedAt	LocalDateTime	None	Auto-updated	Timestamp when achievement claim was last modified.

Table 4: Wellness Check-ins Collection (wellness_checkins)
Description: Captures daily subjective wellness ratings to monitor athlete recovery and calculate readiness scores.

Field Name	Data Type	Key / Index	Null / Constraint	Description
id	String (ObjectId)	Primary Key (@Id)	Not Null, Unique	Unique check-in identifier.
userId	String	Foreign Key (users.id)	Not Null, Compound Index	Athlete user ID.
sleepQuality	Integer	None	Scale: 1 to 5	Subjective sleep score (1 = Insomnia, 5 = Restful).

Field Name	Data Type	Key / Index	Null / Constraint	Description
muscleSoreness	Integer	None	Scale: 1 to 5	Soreness rating (1 = None, 5 = Severe soreness).
energyLevel	Integer	None	Scale: 1 to 5	Energy level (1 = Exhausted, 5 = Peak energy).
readinessScore	Integer	None	Scale: 0 to 100 (%)	Algorithmic readiness metric based on survey inputs.
notes	String	None	Nullable	Daily notes on pain, fatigue, or illness.
date	LocalDate	Compound Index	Not Null	Date of survey (one record per athlete per day).
createdAt	LocalDateTime	None	Auto-generated	Exact timestamp of survey submission.

Table 5: Trainings Collection (trainings)

Description**:** Stores coach-prescribed workout assignments, drill instructions, and athlete feedback.

Field Name	Data Type	Key / Index	Null / Constraint	Description
id	String (ObjectId)	Primary Key (@Id)	Not Null, Unique	Unique workout session identifier.
coachId	String	Foreign Key (users.id)	Not Null, Indexed	Prescribing coach identifier.
athleteId	String	Foreign Key (users.id)	Not Null, Indexed	Assigned athlete identifier.
title	String	None	Not Null	Training session title (e.g., "Explosive Leg Power Routine").
description	String	None	Not Null	Step-by-step drill instructions, sets, reps, and tempos.
category	String	None	Not Null	Category (Strength, Speed, Recovery, Mobility).
date	LocalDate	Secondary Index	Not Null	Scheduled execution date.

Field Name	Data Type	Key / Index	Null / Constraint	Description
time	String	None	Nullable	Target session time (e.g., "07:30 AM").
status	String (Enum)	None	Default: SCHEDULED	State (SCHEDULED, COMPLETED, CANCELLED).
rpe	Integer	None	Scale: 1 to 10	Athlete's Rating of Perceived Exertion post-workout.
actualDurationMinutes	Integer	None	Nullable	Recorded duration spent completing session.
athleteFeedback	String	None	Nullable	Post-workout athlete comments on session fatigue.
completedAt	LocalDateTime	None	Nullable	Timestamp when athlete finalized session.

Table 6: Coach-Athlete Assignments Collection (coach_athlete_assignments)

Description**:** Represents relational pairings between certified coaches and assigned athlete rosters.

Field Name	Data Type	Key / Index	Null / Constraint	Description
id	String (ObjectId)	Primary Key (@Id)	Not Null, Unique	Unique assignment identifier.
coachId	String	Foreign Key (users.id)	Not Null, Indexed	ID of the assigned coach.
athleteId	String	Foreign Key (users.id)	Not Null, Indexed	ID of the assigned athlete.
assignedAt	LocalDateTime	None	Auto-generated	Timestamp when pairing was formalized.
status	String (Enum)	None	Default: ACTIVE	Current state (ACTIVE, PENDING, TERMINATED).

Table 7: Chat Messages Collection (chat_messages)

Description**:** Stores direct 1-on-1 messages between coaches and athletes.

Field Name	Data Type	Key / Index	Null / Constraint	Description
id	String (ObjectId)	Primary Key (@Id)	Not Null, Unique	Unique message identifier.
conversationId	String	Compound Index	Not Null	Composite conversation key (e.g., athleteId_coachId).
senderId	String	Foreign Key (users.id)	Not Null	User ID of message author.
senderName	String	None	Not Null	Denormalized display name of sender.
senderRole	String	None	Not Null (ATHLETE, COACH)	Role of author.
recipientId	String	Foreign Key (users.id)	Not Null	User ID of designated recipient.
content	String	None	Not Null	Text body of the message.
messageType	String	None	Default: TEXT	Classification (TEXT, SYSTEM, MEDIA, TRAINING_LINK).
attachmentUrl	String	None	Nullable	Link to uploaded training media or documents.
isRead	Boolean	None	Default: false	Read receipt indicator.
timestamp	LocalDateTime	Compound Index	Auto-generated	Date and time message was sent.

Table 8: Notifications Collection (notifications)

Description**:** Stores system alerts, achievement approval notifications, and training reminders.

Field Name	Data Type	Key / Index	Null / Constraint	Description
id	String (ObjectId)	Primary Key (@Id)	Not Null, Unique	Unique notification identifier.

Field Name	Data Type	Key / Index	Null / Constraint	Description
userId	String	Foreign Key (users.id)	Not Null, Indexed	Recipient user ID.
title	String	None	Not Null	Alert headline.
message	String	None	Not Null	Comprehensive notification message text.
type	String	None	Not Null	Alert category (VERIFICATION, TRAINING, SYSTEM).
isRead	Boolean	Secondary Index	Default: false	Status indicating if user viewed notification.
link	String	None	Nullable	Deep link route for client-side navigation (e.g., /achievements).
referenceType	String	None	Nullable	Referenced model (TRAINING, ACHIEVEMENT).
referenceId	String	None	Nullable	ID of associated model record.
createdAt	LocalDateTime	Secondary Index	Auto-generated	Timestamp when notification was generated.

Table 9: Admin Audit Logs Collection (admin_audit_logs)
Description: Immutable governance audit log documenting administrative interventions.

Field Name	Data Type	Key / Index	Null / Constraint	Description
id	String (ObjectId)	Primary Key (@Id)	Not Null, Unique	Unique audit entry identifier.
action	String	Secondary Index	Not Null	Standardized code (e.g., USER_ROLE_UPDATED, USER_DELETED).
performedByAdminId	String	Foreign Key (users.id)	Not Null	User ID of executing administrator.
performedByAdminName	String	None	Not Null	Display name of acting administrator.
targetEntityId	String	None	Not Null	Identifier of affected user, roster, or setting.
targetEntityName	String	None	Nullable	Display name of affected entity.

Field Name	Data Type	Key / Index	Null / Constraint	Description
details	String	None	Not Null	Granular description of changes applied.
timestamp	LocalDateTime	Secondary Index	Immutable (now())	Exact timestamp of administrative action.

Table 10: Athlete Goals Collection (goals)

Description: Tracks target athletic benchmarks, deadlines, and milestone completion status.

Field Name	Data Type	Key / Index	Null / Constraint	Description
id	String (ObjectId)	Primary Key (@Id)	Not Null, Unique	Unique goal identifier.
userId	String	Foreign Key (users.id)	Not Null, Indexed	Athlete identifier.
title	String	None	Not Null	Goal title (e.g., "Sub-11 Second 100m Sprint").
category	String	None	Not Null	Metric area (SPEED, STRENGTH, ENDURANCE).
targetValue	Double	None	Not Null	Target benchmark value.
currentValue	Double	None	Not Null	Starting or current baseline value.
unit	String	None	Not Null	Unit of measurement (seconds, kg, km, reps).
targetDate	LocalDate	None	Not Null	Deadline date to achieve target.
completed	Boolean	None	Default: false	Goal completion flag.
createdAt	LocalDateTime	None	Auto-generated	Timestamp when goal was established.

Table 11: System Activities Collection (activities)

Description: Activity stream recording athlete timeline events, badge unlocks, and workouts.

Field Name	Data Type	Key / Index	Null / Constraint	Description
id	String (ObjectId)	Primary Key (@Id)	Not Null, Unique	Unique activity identifier.

Field Name	Data Type	Key / Index	Null / Constraint	Description
userId	String	Foreign Key (users.id)	Not Null, Indexed	ID of the athlete whose action triggered activity.
type	String	None	Not Null	Activity classification (e.g., DRILL_LOG, BADGE_EARNED).
title	String	None	Not Null	Headline summary of event.
description	String	None	Nullable	Additional context regarding event.
timestamp	LocalDateTime	Secondary Index	Auto-generated	Event occurrence timestamp.
icon	String	None	Nullable	Identifier of icon to display in feed.

Table 12: Coach Connection Requests Collection (coach_connection_requests)

Description: Tracks athlete requests sent to coaches to join their training squad.

Field Name	Data Type	Key / Index	Null / Constraint	Description
id	String (ObjectId)	Primary Key (@Id)	Not Null, Unique	Unique request identifier.
athleteId	String	Foreign Key (users.id)	Not Null, Indexed	ID of requesting athlete.
athleteName	String	None	Not Null	Athlete full name.
athleteSport	String	None	Nullable	Athlete sport discipline.
coachId	String	Foreign Key	Not Null, Indexed	Target coach ID.

Field Name	Data Type	Key / Index	Null / Constraint	Description
		(users.id)		
coachName	String	None	Not Null	Coach full name.
message	String	None	Nullable	Personal message from athlete.
status	String	None	Default: PENDING	Status (PENDING, ACCEPTED, DECLINED).
createdAt	LocalDateTime	None	Auto-generated	Request dispatch timestamp.

Table 13: Performance History Collection (performance_history)

Description: Time-series snapshot snapshots used to render historical progression curves.

Field Name	Data Type	Key / Index	Null / Constraint	Description
id	String (ObjectId)	Primary Key (@Id)	Not Null, Unique	Unique history entry identifier.
userId	String	Foreign Key (users.id)	Not Null, Compound Index	Athlete identifier.
speed	Double	None	Not Null	Historical speed rating (0–100).
strength	Double	None	Not Null	Historical strength rating (0–100).
endurance	Double	None	Not Null	Historical endurance rating (0–100).
agility	Double	None	Not Null	Historical agility rating (0–100).
overallScore	Double	None	Not Null	Aggregate score at that time point.
recordedAt	LocalDateTime	Compound Index	Not Null	Snapshot timestamp.

Table 14: Coach Evaluations Collection (coach_evaluations)

Description: Professional coach assessment reviews, target scores, and athlete readiness ratings.

Field Name	Data Type	Key / Index	Null / Constraint	Description
id	String (ObjectId)	Primary Key (@Id)	Not Null, Unique	Unique evaluation identifier.
coachId	String	Foreign Key (users.id)	Not Null, Indexed	Reviewing coach identifier.
athleteId	String	Foreign Key (users.id)	Not Null, Indexed	Evaluated athlete identifier.
readinessStatus	String	None	Not Null	Status (READY, NEEDS_IMPROVEMENT, FATIGUED, INJURY_RISK).
coachFeedback	String	None	Not Null	Formal qualitative critique.
targetSpeed	Double	None	Nullable	Coach-assigned benchmark speed target.
targetStrength	Double	None	Nullable	Target strength score.
targetEndurance	Double	None	Nullable	Target endurance score.
targetAgility	Double	None	Nullable	Target agility score.

Field Name	Data Type	Key / Index	Null / Constraint	Description
createdAt	LocalDateTime	None	Auto-generated	Evaluation creation timestamp.
updatedAt	LocalDateTime	None	Auto-updated	Evaluation revision timestamp.

CHAPTER 5

SYSTEM TESTING

5.1 INTRODUCTION

System testing is an essential verification and validation phase in the software development lifecycle that evaluates the complete, integrated **AthleteX** platform against its functional specifications and quality benchmarks. The primary objective is to identify defects, verify boundary conditions, validate security filters, and ensure high availability before production deployment. In AthleteX, testing verifies not only individual algorithmic routines—such as fatigue score calculations and password hashing—but also complex end-to-end user journeys encompassing AI workout generation, coach achievement verification, real-time messaging, and scannable QR-code credential validation.

5.2 TEST PLAN

The AthleteX test plan establishes a structured, multi-tiered testing strategy designed to achieve comprehensive code coverage and operational reliability:

- **Scope:** Covers backend business logic, REST API controller contracts, database persistence, frontend React component lifecycles, and automated cross-browser user flows.
- **Testing Environment:** Java 21, Spring Boot Test runner, MongoDB local/test instance, Node.js with JSDOM, and Selenium WebDriver with Headless Google Chrome.
- **Pass Criteria:** 100% execution success on all critical test paths with zero unhandled exceptions, zero data leaks across role boundaries, and deterministic assertions across all suites.

5.2.1 Unit Testing

Unit testing validates the smallest testable units of source code—individual classes, algorithms, and service methods—in complete isolation from external dependencies:

- **Backend Unit Tests (JUnit 5 & Mockito):** Business services are tested by isolating them from MongoDB using Mockito mocks. Key test classes include:
    - AuthServiceTest: Verifies user registration, password strength enforcement (minimum 8 characters, uppercase, digit, special character), duplicate email prevention, and credential hashing.
    - BadgeServiceTest: Validates badge-unlock algorithms, verifying that achievement claims correctly award badges such as the *First Step Trophy* or *Coach Verified Champion*.

    - CoachReportServiceTest: Confirms performance metric aggregation (speed, strength, endurance) and asserts that coaches cannot access telemetry of unassigned athletes.
- **Frontend Unit Tests (Vitest & React Testing Library):** Validates isolated UI components and state machines. Key tests include Login.test.jsx for input validation and state transitions, and ProtectedRoute.test.jsx for unauthorized redirection logic.

5.2.2 Integration Testing

Integration testing verifies that independently developed modules interact correctly across their programmatic interfaces, data transport layers, and protocol boundaries:

- **REST API Controller Contracts (Spring MockMvc):** Validates HTTP request/response mappings, JSON serialization, and status codes without requiring a live web server. Key suites include:
    - AuthControllerTest: Confirms that POST /api/auth/register and POST /api/auth/login accept valid DTO payloads, invoke security providers, and return correctly structured JSON Web Tokens (JWT).
    - NotificationControllerTest: Asserts that GET /api/notifications and PUT /api/notifications/{id}/read correctly fetch user alerts and update unread statuses in the database.
- **Database & Security Integration:** Validates that Spring Security interceptors correctly parse incoming Bearer JWT tokens, inject user authentication into the Security Context, and enforce Role-Based Access Control (**RBAC**) across /api/athlete/**, /api/coach/**, and /api/admin/** endpoints.

5.2.3 Validation Testing or System Testing

Validation (or System) testing verifies the integrated software against the complete Software Requirements Specification (SRS) to ensure that business goals are fully realized:

End-to-End Workflow Validation: Executes end-to-end functional flows, including:

An athlete logging a drill, receiving an AI-generated conditioning regimen, and submitting an achievement with video proof.

An assigned coach reviewing the pending claim, inspecting proof, and accrediting the achievement.

A public recruiter accessing the verification URL (/verify/:id) to confirm the validated credential.

Boundary and Exception Testing: Evaluates system resilience under abnormal inputs, such as submitting out-of-range fatigue scores, invalid OTPs, malformed JSON bodies, or unauthorized cross-roster data requests, asserting that clean, descriptive HTTP error codes (400, 401, 403, 404) are returned.

5.2.4 Output Testing or User Acceptance Testing

Output testing and User Acceptance Testing (UAT) assess the correctness of generated business deliverables and confirm that the platform meets the practical expectations of its end users:

- **Output Data Fidelity & Export Verification:**
  - **PDF Performance Cards:** Verifies that official progress reports compiled via jsPDF and jspdf-autotable accurately format coach evaluations, table alignments, and aggregate scores into printable documents.
  - **QR Code Readability:** Asserts that scannable QR codes generated on athlete profiles resolve cleanly to active public verification URLs across standard mobile cameras and barcode scanners.
  - **Visual Chart Rendering:** Validates that Recharts radar and line charts accurately plot multi-axis telemetry without clipping or distortion across varying screen resolutions.
- **User Acceptance Testing (UAT):** Conducted with representative Athletes, Coaches, and Administrators who execute common operational

5.2.5 Automation Testing

Automation testing establishes a repeatable, continuous quality assurance pipeline that eliminates human error and validates code integrity across every build:

- **Continuous Test Execution:** Automated test suites are integrated into the build lifecycles of both backend (mvn test) and frontend (npm test / Vitest).
- **Determinism & Speed:** Unit and integration tests execute in memory within seconds, providing immediate feedback during code refactoring and continuous integration (CI) workflows.
- **Metrics:** AthleteX maintains a verified **100% automated test pass rate** (23 out of 23 test suites passing) across all business logic, controllers, UI components, and browser integration targets.

5.2.6 Selenium Testing

Selenium testing validates real-world user interactions by automating live browser sessions against the rendered Single Page Application:

- **Framework & Architecture:** Built with **Selenium WebDriver 4**, WebDriverManager for automated ChromeDriver binary management, and Headless Google Chrome to simulate real browser environments without graphical overhead.
- **Test Class (AthleteXWebE2ETest):**
    - testHomepageLoads: Launches headless Chrome, navigates to http://localhost:5173, verifies page title metadata, and uses explicit WebDriver waits to confirm that the React DOM root (#root) container mounts successfully.
    - testLoginPageControls: Navigates to the /login route, asserts the presence of the authentication card container, validates the email and password input fields, verifies sign-in call-to-action buttons, and asserts that branding elements render accurately.
- **Lifecycle Management:** Utilizes automated @BeforeAll setup to initialize browser capabilities and @AfterAll teardown hooks to safely terminate the browser process, preventing background resource leaks.

AUTOMATE TESTCASE REPORT

1. Backend Unit Test Cases (JUnit 5 & Mockito)

1.1 Test Class: AuthServiceTest

Test Case ID	Test Scenario / Description	Input / Test Data	Expected Result	Actual Result	Status
TC-AUTH-01	Verify successful registration of a new user account with encrypted password storage.	RegisterRequest: Full Name: "John Runner", Email: "john@athletex.com", Password: "Password123!", Role: ATHLETE	Password is encrypted with BCrypt, user is saved to repository, and confirmation string is returned.	User created and saved with encoded password; returns confirmation.	**PASS**
TC-AUTH-02	Verify rejection of user registration when attempting to use an already registered email.	RegisterRequest: Existing email "john@athletex.com"	IllegalArgumentException thrown with message indicating email already in use; no duplicate user created.	Exception thrown: "Email is already registered"; duplicate prevented.	**PASS**
TC-AUTH-03	Verify password complexity validation policy enforcement during registration.	RegisterRequest: Weak password "pass" (< 8 characters, no special character)	IllegalArgumentException thrown; registration rejected due to weak password policy.	Exception thrown enforcing minimum 8 chars, uppercase, digit, and special character.	**PASS**
TC-AUTH-04	Verify successful user login with valid credentials and	LoginRequest: Email: "john@athletex.com", Password: "Password123!"	Authentication verified, activity log entry created, and valid JWT token returned in LoginResponse.	JWT token issued; user role ATHLETE returned; login event logged.	**PASS**

Test Case ID	Test Scenario / Description	Input / Test Data	Expected Result	Actual Result	Status
	activity logging.				
TC-AUTH-05	Verify login rejection when an unregistered email address is submitted.	LoginRequest: Unregistered email "ghost@athletex.com", Password: "Password123!"	BadCredentialsException or user not found exception thrown; access denied.	Exception thrown indicating invalid credentials; no token issued.	**PASS**
TC-AUTH-06	Verify login rejection when an incorrect password is provided for an existing account.	LoginRequest: Email: "john@athletex.com", Incorrect Password: "WrongPassword99!"	Authentication fails; exception thrown; zero security token generated.	Access rejected with bad credentials response; no JWT generated.	**PASS**

1.2 Test Class: BadgeServiceTest

Test Case ID	Test Scenario / Description	Input / Test Data	Expected Result	Actual Result	Status
TC-BDG-01	Verify that all achievement badges remain locked when an athlete has logged zero achievements.	Athlete ID: "ath_01", Achievement count: 0	Evaluated badge list returns all badges with status locked (unlocked = false).	All achievement badges remain locked.	**PASS**
TC-BDG-02	Verify automatic unlocking of "First Step Trophy" badge upon logging first athletic achievement.	Athlete ID: "ath_01", Achievement count: 1 (Category: "Fitness")	"First Step Trophy" badge status changes to unlocked (unlocked = true).	"First Step Trophy" badge awarded and unlocked successfully.	**PASS**

Test Case ID	Test Scenario / Description	Input / Test Data	Expected Result	Actual Result	Status
TC-BDG-03	Verify unlocking of "Gold Standard" and "Coach Verified Champion" badges upon verified awards.	Athlete ID: "ath_01", Achievement with Level: "State", Category: "Gold", isVerified = true	Unlocks "Gold Standard", "Coach Verified Champion", and "Verified Media Proof" badges.	All 3 advanced badges unlocked and updated in athlete profile.	**PASS**

1.3 Test Class: CoachReportServiceTest

Test Case ID	Test Scenario / Description	Input / Test Data	Expected Result	Actual Result	Status
TC-RPT-01	Verify compilation of individual athlete performance metrics and longitudinal trend lines.	Coach ID: "coach_01", Assigned Athlete ID: "ath_01" (Active Pairing)	Returns aggregated score, speed, strength, endurance, agility radar metrics, and trend history.	Report generated containing all biometric radar scores and trend points.	**PASS**
TC-RPT-02	Enforce security isolation so coaches cannot access performance reports of unassigned athletes.	Coach ID: "coach_01", Unassigned Athlete ID: "ath_99"	SecurityException or access denied response; report compilation blocked.	Access denied; unauthorized cross-roster data access prevented.	**PASS**

2. REST API Controller Integration Test Cases (Spring MockMvc)

2.1 Test Class: AuthControllerTest

Test Case ID	Test Scenario / Description	HTTP Request / Endpoint	Expected Result	Actual Result	Status
TC-API-AUTH-01	Verify HTTP 200 response upon posting a valid user registration payload.	POST /api/auth/register Payload: Valid RegisterRequest JSON	HTTP Status 200 (OK); String confirmation message returned in response body.	HTTP 200 OK returned with registration confirmation string.	**PASS**

Test Case ID	Test Scenario / Description	HTTP Request / Endpoint	Expected Result	Actual Result	Status
TC-API-AUTH-02	Verify HTTP 200 response returning a valid JWT token payload upon valid credentials.	POST /api/auth/login Payload: Valid LoginRequest JSON	HTTP Status 200 (OK); JSON response containing token, role, and userId.	HTTP 200 OK returned with JSON body containing signed JWT token.	**PASS**

2.2 Test Class: NotificationControllerTest

Test Case ID	Test Scenario / Description	HTTP Request / Endpoint	Expected Result	Actual Result	Status
TC-API-NOTIF-01	Verify fetching notifications for a specific user identifier.	GET /api/notifications?userId=user_01 Header: Bearer JWT Token	HTTP Status 200 (OK); Array of notification objects for that user.	HTTP 200 OK returned with list of user notifications.	**PASS**
TC-API-NOTIF-02	Verify retrieval of unread notification count for an authenticated user.	GET /api/notifications/unread-count?userId=user_01 Header: Bearer JWT Token	HTTP Status 200 (OK); Integer representing unread alert count.	HTTP 200 OK returned with accurate unread notification count.	**PASS**
TC-API-NOTIF-03	Verify updating notification status from unread to read.	PUT /api/notifications/{id}/read Header: Bearer JWT Token	HTTP Status 200 (OK); Notification document updated with isRead = true.	HTTP 200 OK returned; notification marked read in MongoDB.	**PASS**

3. Selenium E2E Browser Automation Test Cases (Selenium WebDriver 4)

Test Class: AthleteXWebE2ETest

Test Case ID	Test Scenario / Description	Test Steps / Actions	Expected Result	Actual Result	Status
TC-E2E-01	Verify homepage loads cleanly and mounts the React root DOM container.	1. Launch Headless Google Chrome (1920x1080). 2. Navigate to http://localhost:5173. 3. Poll for DOM element #root using WebDriverWait (10s).	Document title is non-null; #root container mounts successfully with zero script crash.	#root element located; homepage loaded within timeout.	**PASS**
TC-E2E-02	Verify navigation to login route and presence of branding and form controls.	1. Navigate to http://localhost:5173/login. 2. Wait for HTML <body> container. 3. Assert page source contains branding and form controls.	Page renders authentication form with "AthleteX" branding and "Sign In" elements.	"AthleteX" branding and authentication inputs detected in DOM.	**PASS**

4. Frontend Component Test Cases (Vitest & React Testing Library)

4.1 Test Class: ProtectedRoute.test.jsx

Test Case ID	Test Scenario / Description	Simulated Context / State	Expected Result	Actual Result	Status
TC-FE-ROUTE-01	Verify that unauthenticated users are redirected to the /login route.	Unauthenticated user (Token: null) attempting to access /dashboard.	Child component is blocked; browser redirects to /login.	User redirected to /login; protected view blocked.	**PASS**
TC-FE-ROUTE-02	Verify that authorized users with matching role can access protected child view.	Authenticated user (Role: ATHLETE) accessing /athlete/dashboard.	Protected child component renders into the DOM successfully.	Child component rendered with matching role.	**PASS**
TC-FE-ROUTE-03	Verify role-based redirection when user role does not match required route role.	Authenticated user (Role: ATHLETE) attempting to open /admin/users.	Access to admin view blocked; user redirected to	Access blocked; user redirected to appropriate	**PASS**

Test Case ID	Test Scenario / Description	Simulated Context / State	Expected Result	Actual Result	Status
			athlete dashboard.	role dashboard.	

4.2 Test Class: Login.test.jsx

Test Case ID	Test Scenario / Description	User Actions / Events	Expected Result	Actual Result	Status
TC-FE-UI-01	Verify correct rendering of sign-in header, input fields, and submit button.	Render <Login /> component inside JSDOM environment.	Form contains "Sign In" header, email input, password input, and submit button.	All form elements located with proper labels and placeholders.	**PASS**
TC-FE-UI-02	Verify user input handling and real-time state synchronization.	Simulate typing " test@athletex.com" into email field and "Secret123!" into password field.	Input field values update dynamically to reflect user keystrokes.	Component state updated; input values match typed text.	**PASS**

5. Consolidated Test Execution Summary Table

Category	Testing Framework	Test Suite / Class Name	Total Tests	Passed	Failed	Pass Rate
Backend Service Unit	JUnit 5, Mockito	AuthServiceTest	6	6	0	100%
Backend Service Unit	JUnit 5, Mockito	BadgeServiceTest	3	3	0	100%
Backend Service Unit	JUnit 5, Mockito	CoachReportServiceTest	2	2	0	100%
REST Controller Integration	Spring MockMvc	AuthControllerTest	2	2	0	100%
REST Controller Integration	Spring MockMvc	NotificationControllerTest	3	3	0	100%
Browser E2E Automation	Selenium WebDriver 4, Chrome	AthleteXWebE2ETest	2	2	0	100%
Frontend Route Security	Vitest, React Testing Library	ProtectedRoute.test.jsx	3	3	0	100%

Category	Testing Framework	Test Suite / Class Name	Total Tests	Passed	Failed	Pass Rate
Frontend Form Component	Vitest, React Testing Library	Login.test.jsx	2	2	0	100%
OVERALL TOTAL			**23**	**23**	**0**	**100%**

CHAPTER 6

IMPLEMENTATION

6.1 INTRODUCTION

Implementation refers to the process of putting into action or carrying out the planned activities and strategies to achieve the project's objectives. It involves the execution of various tasks, activities, and processes outlined in the project plan, such as designing, building, testing, and deploying a product or service. Implementation typically follows the planning phase, where the project team has defined the project scope, objectives, and deliverables, and created a detailed project plan. During implementation, the team puts the plan into action, using the necessary resources, such as technology, people, and finances, to execute the project activities. Implementation in a project involves the practical realization of the project plan, including the execution of various activities and processes, as well as ongoing monitoring and control to ensure successful project completion.

6.2 IMPLEMENTATION PROCEDURES

"Implementation procedures" refer to a set of actions, steps, and processes that a project team follows to execute a project plan and achieve its goals. It involves breaking down the project plan into smaller, manageable tasks and allocating resources to each task. The team also establishes communication channels to keep everyone informed of project progress, monitors the project's progress, and takes corrective action if necessary. Finally, the team tests the project outcomes and deploys the final product or service to the end-users. The main goal of implementation procedures is to ensure that the project objectives are met on time, within budget, and with the required quality standards.

6.2.1 User Training

User training is an essential component of implementation procedures that involves providing instruction and guidance to end-users on how to use a new product or system. It is a critical step in ensuring the success of a project as it enables users to understand how to use the product or service effectively and efficiently. Usertraining is very important in astage at which primary objective of user training is to equip users with the necessary knowledge, skills, and confidence to use the product or system to achieve the desired outcomes. User training is a crucial element of implementation procedures that involves educating end users on how to use a new product or system effectively.

6.2.2 Training on the Application Software

This is an effective step in which the end users are trained with how the software application should run with. It refers to the process of providing instruction and guidance to the end-users or stakeholders on how to use the application software effectively. The training may cover various aspects of the application software, such as its features, functionalities, and user interface. Effective training on the application software should be customized to the needs and skills of the end-users. The training should be interactive and hands-on, allowing the users to practice and apply the skills learned. The training should also be evaluated to measure its effectiveness and identify areas for improvement.

6.2.3 System Maintenance

System maintenance is an effective component such that System maintenance refers to the ongoing activities required to ensure that a system or application operates effectively and efficiently after it has been implemented. It involves regular updates, bug fixes, and performance optimizations to keep the system running smoothly and securely. System maintenance is essential to ensure that a system remains operational and effective after implementation. By establishing maintenance procedures and following them consistently, project teams can ensure that the system operates smoothly, remains secure, and continues to meet the needs of the end-users.

6.2.4 Hosting

Hosting is the generic term that we used. It mean that hosting is mainly used for storing and securing our project in platform. Web hosting is simply means that hosting of a website. So that the project that we stored and maintained through the local system can be made possible through the online web hosting platforms. It enables the user to access the website over the internet.

Procedure for hosting a website:

1. Introduction to Containerized Deployment with Docker

To guarantee reliable execution across diverse host environments and eliminate configuration drift between local development and cloud production, the AthleteX platform is fully containerized using Docker and orchestrated via Docker Compose. Containerization encapsulates each architectural layer—the MongoDB database, the Java 21 / Spring Boot 3 backend application, and the React 19 / Vite frontend Single Page Application—along with its exact runtime dependencies, system binaries, libraries, and environment configurations into isolated, lightweight execution units called containers.

This containerized approach ensures that the entire AthleteX ecosystem behaves identically whether executed on a developer's local workstation, deployed within a cloud provider like Render, or hosted on a dedicated Linux virtual private server (VPS). By leveraging multi-stage builds, the deployment minimizes production image footprints, optimizes attack surfaces by utilizing non-root runtime users, and enables single-command startup of the entire multi-service architecture.

---

2. Backend Containerization Strategy (Multi-Stage Dockerfile)

The backend service is packaged using a multi-stage Dockerfile located within the athletex-backend directory, strategically dividing the build process into two distinct phases to achieve speed, security, and minimal image size.

In the first stage, designated as the builder stage, a lightweight Maven image based on Alpine Linux and Eclipse Temurin JDK 21 (maven:3.9.6-eclipse-temurin-21-alpine) is initialized. To take full advantage of Docker's layer-caching mechanism, the Maven project descriptor (pom.xml) is copied into the container first, and all third-party dependencies are pre-downloaded offline using mvn dependency:go-offline -B. Subsequently, the application source code is copied into the container, and the production executable JAR artifact is compiled and packaged via mvn clean package -DskipTests, deliberately skipping automated unit test execution to accelerate image assembly during production deployments.

In the second stage, designated as the production runtime, a stripped-down Alpine Java Runtime Environment (eclipse-temurin:21-jre-alpine) is instantiated without the heavy Maven build tools or compilers. To comply with modern enterprise security and least-privilege standards, a dedicated non-root system group and user (athletex:athletex) are created, ensuring the application does not execute with root administrative privileges. The compiled executable artifact (athletex-backend-0.0.1-SNAPSHOT.jar) is extracted from the builder stage into the runtime environment as app.jar. The container exposes network port 8080 and defines an entrypoint command that launches the application using java -jar app.jar, resulting in a highly optimized, production-ready container image under 250 megabytes.

---

3. Frontend Containerization Strategy (Multi-Stage Dockerfile and Nginx)

The frontend client is similarly containerized using a multi-stage Dockerfile situated in athletex-frontend, combining a Node.js compilation environment with an ultra-lightweight Nginx web server.

In the initial build stage, an official Node.js 20 environment on Alpine Linux (node:20-alpine) is established. Build arguments, including VITE_API_BASE_URL, are ingested and converted into

environment variables to ensure that the compiled JavaScript bundles point to the correct backend API endpoint. The package manifests (package.json and package-lock.json) are copied into the workspace, and dependencies are cleanly installed using npm ci. The React source code, Tailwind CSS styling tokens, and static assets are then injected, and the production compilation script npm run build is executed to generate an optimized, minified distribution directory (dist).

In the second stage, an official, minimal Nginx image on Alpine Linux (nginx:alpine) is initialized to serve the compiled client application. The production web assets from the builder stage are copied directly into the web root directory /usr/share/nginx/html. Furthermore, a tailored Nginx configuration file (nginx.conf) is copied into the web server settings. This configuration activates Gzip compression across JavaScript, CSS, and JSON resources, enforces public browser caching for static media assets, and implements client-side Single Page Application (SPA) routing fallbacks via the directive try_files $uri $uri/ /index.html. This ensures that refreshing nested routes such as /dashboard or /verify/:id does not trigger HTTP 404 errors. Additionally, the Nginx configuration establishes a reverse proxy rule forwarding all requests directed to /api/ to the backend container on port 8080, facilitating unified domain routing on exposed web port 80.

---

4. Multi-Service Orchestration via Docker Compose

To coordinate the lifecycle, dependencies, networking, and storage across all components, the platform utilizes a centralized orchestration file (docker-compose.yml) located in the project root directory.

The orchestration file declares three interconnected services: mongodb, backend, and frontend. The mongodb service pulls the official mongo:7.0 image, exposing port 27017 and mounting a named volume mongodb_data to ensure all athletic performance logs, user credentials, and chat messages persist safely on the host machine across container restarts. It incorporates an automated internal health check running mongosh --eval "db.adminCommand('ping')" every ten seconds. The backend service builds from the backend Dockerfile, exposes port 8080, and specifies environment variables connecting it to the database at mongodb://mongodb:27017/athletex_db. Crucially, it includes a depends_on condition requiring the MongoDB container to be fully healthy before launching the Spring Boot backend, preventing connection failure crashes during system startup. The frontend service builds from the frontend Dockerfile, exposes port 80, and waits for the backend service to initialize. All three services communicate over an isolated private bridge network named athletex-network, allowing internal inter-container communication using container hostnames rather than volatile IP addresses.

---

5. Step-by-Step Procedure for Local and Cloud VPS Deployment

Deploying the complete AthleteX application using Docker on a local development machine or a cloud virtual machine (such as an AWS EC2 instance or an Ubuntu VPS) follows a systematic, five-step workflow:

First, the target host machine is prepared by ensuring that Docker Engine and the Docker Compose plugin are installed and running with active system daemon permissions. The host machine's security groups or firewall rules must allow inbound traffic on port 80 for web traffic, port 8080 for API interactions, and port 22 for secure administrative SSH access.

Second, the complete AthleteX codebase is cloned from the remote version control repository onto the host server using git clone https://github.com/<your-username>/AthleteX.git, and the working directory is shifted into the project root.

Third, the production environment file is established. The developer copies the provided template .env.example into an active .env file using the command cp .env.example .env. Within this file, production values are verified—including email transmission credentials (MAIL_USERNAME and MAIL_PASSWORD), the database connection target, and the public-facing API address passed into VITE_API_BASE_URL.

Fourth, the entire multi-container architecture is built and launched simultaneously in the background by executing the single orchestration command: docker-compose up -d --build. Docker Compose automatically pulls the base images, builds the multi-stage backend and frontend images, provisions the isolated bridge network and volume storage, and starts all containers in their designated dependency order.

Fifth, the system deployment status is verified. The administrator runs docker-compose ps to inspect running container states, confirming that all three services display healthy and running statuses. Real-time application logs can be monitored by running docker-compose logs -f. Accessing the host machine's public IP address or configured domain name in any web browser loads the AthleteX web application, confirming a successful, self-contained container deployment.

---

6. Integrating Docker with Cloud Platforms (MongoDB Atlas and Render)

The containerized architecture also integrates seamlessly with modern cloud hosting platforms, specifically when combining MongoDB Atlas with Render.

When deploying the backend to Render as a Web Service, Render can be instructed to build directly from the backend Dockerfile rather than using a generic runtime environment. The developer points Render to the repository, specifies the root directory as athletex-backend, and selects Docker as the environment. In the Render environment configuration, the local database connection variable is

overridden with the external cloud connection string: MONGODB_URI=mongodb+srv://<username>:<password>@cluster-address.mongodb.net/athletex_db. During deployment, Render executes the multi-stage Docker build, generating the identical secured Alpine container image and exposing port 8080 via HTTPS. Similarly, if the frontend is hosted as a Dockerized service on Render or any cloud provider, the build argument VITE_API_BASE_URL is passed with the cloud backend URL (such as https://athletex-backend.onrender.com/api). Nginx serves the compiled SPA assets on port 80 with all routing rewrite rules and Gzip compressions fully intact. This hybrid container-cloud methodology gives organizations maximum flexibility: they can run the entire self-contained stack on a single Docker server for offline athletic facilities or deploy the individual containers onto managed cloud services like Render and MongoDB Atlas for global enterprise scalability.

Final Verification

- Access the live application through your custom domain.
- Perform end-to-end testing of all user roles and functionalities
- Verify that updates deploy successfully when pushing new images to Docker Hub

Hosted Link: https://athletex-frontend.onrender.com

Hosted Link QR Code:

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 CONCLUSION

The development and implementation of **AthleteX** successfully address the long-standing operational, technical, and structural limitations that have historically plagued athletic development and sports performance management. By transitioning away from disjointed physical logbooks, isolated spreadsheets, and informal messaging channels, the platform establishes a unified, cloud-ready digital ecosystem that centralizes the entire athletic lifecycle. Built on a decoupled micro-monolithic architecture utilizing **Java 21**, **Spring Boot 3**, **Spring Data MongoDB**, and **React 19**, AthleteX provides a resilient, high-throughput platform that seamlessly bridges the communication and data divide between athletes, accredited coaches, institutional administrators, and external talent scouts under a zero-trust, Role-Based Access Control (**RBAC**) model.

A key achievement of the platform lies in its fusion of sports science methodologies with modern artificial intelligence and verifiable credentialing. Through direct integration with **Google Gemini Generative AI**, AthleteX replaces static, one-size-fits-all training templates with dynamic, multi-week workout regimens that adapt to an athlete's real-time wellness, daily fatigue indices, and sport-specific milestones. Furthermore, the platform introduces a transparent, tamper-proof credentialing paradigm: athletic achievements are reviewed against attached multimedia proof, certified by accredited coaches, and made instantly verifiable to collegiate recruiters and scouts via scannable **QR Codes** and dedicated public verification portals. This eliminates the vulnerability of forged sports records while providing athletes with an immutable digital portfolio.

The operational stability, security, and performance of AthleteX have been demonstrated through an exhaustive automated testing pyramid—encompassing backend unit logic with **JUnit 5** and **Mockito**, REST controller contracts via **Spring MockMvc**, frontend component lifecycles using **Vitest**, and automated headless browser flows via **Selenium WebDriver 4**—attaining a verified **100% test pass rate**. Complemented by production-grade **Docker** containerization and multi-tier cloud hosting on **Render** and **MongoDB Atlas**, the platform demonstrates enterprise-level reliability, low response latency, and effortless deployment scalability. In conclusion, AthleteX democratizes elite-level sports analytics for high schools, collegiate programs, and grassroots academies, delivering a transformative, data-driven framework that elevates athletic potential and establishes an extensible foundation for future innovations in wearable IoT telemetry and computer-vision form analysis.

7.2 FUTURE SCOPE

The architecture and design of **AthleteX** establish a modular, scalable foundation capable of absorbing emerging sports technologies and data science innovations. While the current platform provides a complete ecosystem for workout management, AI conditioning, coach oversight, and public credential verification, several high-impact enhancements are envisioned to further elevate the platform's analytical depth, automation, and global reach:

1. Real-Time IoT and Wearable Sensor Telemetry Ingestion: Future iterations will integrate direct Bluetooth and cloud API connectors for popular sports wearables and telemetry hardware (such as Garmin, WHOOP, Apple Watch, and Polar chest straps). This will enable automated, continuous ingestion of objective physiological signals—including resting Heart Rate Variability (HRV), continuous pulse rates, blood oxygenation (SpO_2), and GPS speed tracks—eliminating manual entry for cardiovascular metrics and providing coaches with laboratory-grade physical readiness data.

2. Computer Vision and AI-Driven Biomechanical Pose Estimation: By leveraging lightweight client-side computer vision frameworks (such as MediaPipe or TensorFlow.js), the platform can introduce automated exercise form analysis. Athletes will be able to record drill attempts on their smartphone cameras, while the computer vision engine calculates joint angles, bar velocity, squat depths, and sprint cadence in real time. This will allow the system to automatically flag biomechanical imbalances and improper form, drastically reducing the risk of acute training injuries.

3. Predictive Injury Modeling via Advanced Machine Learning: Building upon the longitudinal wellness and drill datasets accumulated in MongoDB, future releases can implement predictive analytics using supervised machine learning algorithms (such as Random Forests or Long Short-Term Memory networks). By analyzing the Acute-to-Chronic Workload Ratio (ACWR) alongside cumulative sleep deficits and subjective muscle soreness ratings, the platform will be capable of forecasting musculoskeletal injury risks days before clinical symptoms emerge, prompting coaches to mandate scheduled deload cycles proactively.

4. Decentralized Credentialing via Blockchain and Soulbound Tokens (SBTs): To expand the public verification engine for international scouting federations and Olympic trial committees, the platform's QR verification records can be anchored onto public, decentralized distributed ledgers (such as Polygon or Ethereum layer-2 networks). Verified milestones and tournament records can be minted as non-transferable Soulbound Tokens (SBTs), providing athletes with an immutable,

globally recognized digital passport of sports achievements that is immune to database server outages or organizational disputes.

5. Native Cross-Platform Mobile Applications with Offline Sync: While the current React 19 Single Page Application is fully responsive across mobile browsers, future development will introduce native mobile applications built using React Native or Flutter. This will provide athletes with offline-first local SQLite caching, allowing them to record drill attempts, timings, and wellness scores in subterranean gym basements or remote rural athletic tracks without active cellular service, with data automatically synchronizing to MongoDB Atlas once network connectivity is restored.

6. Automated Nutritional and Hydration Periodization Module: A comprehensive athletic platform must account for fueling and energy expenditure. Future versions can incorporate an AI-assisted sports nutrition engine that calculates daily caloric, macronutrient, and hydration targets tailored specifically to an athlete's daily training volume, body composition objectives, and impending competition schedules.

7. Talent Recruitment Marketplace and NIL Sponsorship Matching: The platform can evolve into a full-scale talent marketplace connecting verified athletes directly with collegiate recruiters, athletic brand sponsors, and Name, Image, and Likeness (NIL) collectives. Talent scouts will be able to filter athletes based on verified physical benchmarks (e.g., verified sub-4.5s 40-yard dash or specific vertical jump thresholds), enabling data-driven scholarship offerings and sponsorships for emerging athletic talent.

CHAPTER 8

BIBLIOGRAPHY

REFERENCES:

Academic Papers and Sports Science Literature

Gabbett, T. J. (2016). *"The training—injury prevention paradox: should athletes be training smarter and harder?"* British Journal of Sports Medicine, 50(5), pp. 273–280. doi:10.1136/bjsports-2015-095788.

Halson, S. L. (2014). *"Monitoring training load to understand fatigue in athletes."* Sports Medicine, 44(Suppl 2), pp. 139–147. doi:10.1007/s40279-014-0253-z.

Bourdon, P. C., Cardinale, M., Murray, A., et al. (2017). *"International Olympic Committee consensus statement on load in sport and risk of injury."* British Journal of Sports Medicine, 51(12), pp. 929–935.

Fielding, R. T. (2000). *"Architectural Styles and the Design of Network-based Software Architectures."* Doctoral dissertation, University of California, Irvine. [Foundational work on RESTful API architecture].

Fowler, M. (2014). *"Microservices: A definition of this new architectural term."* martinfowler.com. Available at: https://martinfowler.com/articles/microservices.html.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). *"Attention Is All You Need."* Advances in Neural Information Processing Systems (NeurIPS 2017), 30, pp. 5998–6008. [Foundational Transformer architecture powering Generative AI].

Gemini Team, Google (2023). *"Gemini: A Family of Highly Capable Multimodal Models."* arXiv preprint arXiv:2312.11805.

Technical Books and Standards

Walls, C. (2022). *Spring in Action, Sixth Edition.* Manning Publications. ISBN: 978-1617297571. [Comprehensive guide to Spring Boot, Spring Security, and Spring Data].

Chodorow, K. (2013). *MongoDB: The Definitive Guide: Powerful and Scalable Data Storage, Second Edition.* O'Reilly Media. ISBN: 978-1449344689. [NoSQL data modeling, compound indexing, and aggregation pipelines].

Banks, A., and Porcello, E. (2020). *Learning React: Modern Patterns for Developing React Applications, Second Edition.* O'Reilly Media. ISBN: 978-1492051725.

Jones, M. B., Bradley, J., and Sakimura, N. (2015). *"JSON Web Token (JWT)."* RFC 7519, Internet Engineering Task Force (IETF). Available at: https://datatracker.ietf.org/doc/html/rfc7519.

Gamma, E., Helm, R., Johnson, R., and Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software.* Addison-Wesley. ISBN: 978-0201633610.

Burnstein, I. (2003). *Practical Software Testing: A Process-Oriented Approach.* Springer-Verlag New York. ISBN: 978-0387951317. [Software testing life cycle, unit testing, and integration verification].

Official Framework and Documentation References

Spring Framework Documentation (2024). *"Spring Boot Reference Documentation (v3.x)."* VMware Tanzu. Available at: https://docs.spring.io/spring-boot/docs/current/reference/html/.

MongoDB Documentation (2024). *"MongoDB Manual (v7.0) & Data Modeling Guidelines."* MongoDB Inc. Available at: https://www.mongodb.com/docs/manual/.

React Documentation (2024). *"React 19 Core Documentation and Server Components."* Meta Open Source. Available at: https://react.dev/.

W3C WebDriver Specification (2024). *"WebDriver: A Browser Automation Interface."* World Wide Web Consortium (W3C). Available at: https://www.w3.org/TR/webdriver2/.

SeleniumHQ Documentation (2024). *"Selenium WebDriver 4 Documentation and Client Bindings."* Software Freedom Conservancy. Available at: https://www.selenium.dev/documentation/.

Docker Documentation (2024). *"Docker Compose Specification and Multi-Stage Container Builds."* Docker Inc. Available at: https://docs.docker.com/.

Google Cloud AI (2024). *"Google Gemini API Developer Guide and Model Capabilities."* Google Developers. Available at: https://ai.google.dev/docs.

CHAPTER 9

APPENDIX

9.1 Sample Code

Program.cs

9.2 Screen Shots

Home page

Registration

Login

All Services Dashboard

Lost and Found Module

House Rentals Module

9.3 GIT Log

9.4 Certificates